



**Escuela Superior
de Ingeniería y Tecnología**
Universidad de La Laguna

Trabajo de Fin de Grado

Aplicación para la visualización de la distribución de cultivos en Canarias

*Application for the visualization of planting distribution in the
Canary Islands*

Adrián Lima García

San Cristóbal de La Laguna, 8 de julio de 2025

D. **Christopher Expósito Izquierdo**, profesor Ayudante Doctor adscrito al Departamento Ingeniería Informática y de Sistemas de la Universidad de La Laguna como tutor

D. **Israel López Plata**, profesor Ayudante Doctor adscrito al Departamento Ingeniería Informática y de Sistemas de la Universidad de La Laguna como cotutor

C E R T I F I C A N

Que la presente memoria titulada:

"Aplicación para la visualización de la distribución de cultivos en Canarias"

ha sido realizada bajo su dirección por Don **Adrián Lima García**.

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos firman la presente en San Cristóbal de La Laguna a 8 de julio de 2025

Agradecimientos

A mis tutores, Christopher e Israel, quiero darles mi más sincero agradecimiento por su guía, dedicación y apoyo durante esta etapa. Su experiencia y sus consejos han sido fundamentales para poder llevar a cabo este proyecto.

A mi familia, quiero agradecerles por su amor y apoyo durante estos años. Especialmente a mis padres y mi abuela, quienes más de cerca han vivido esta etapa conmigo, por su apoyo en los buenos y en los malos momentos, por animarme siempre a seguir adelante y estar a mi lado en cada paso de este camino. Gracias por todo.

A mis amigos, gracias por estar siempre ahí, por animarme cuando lo necesitaba y acompañarme para celebrar los logros. Por compartir conmigo esta etapa tan importante de mi vida.

Finalmente, quiero agradecer a todas aquellas personas que de alguna forma u otra han formado parte de esta etapa.

Licencia



©Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional.

Resumen

Este Trabajo de Fin de Grado tiene como objetivo el desarrollo de un visualizador de la distribución de cultivos en Canarias. La finalidad principal es proporcionar herramientas que faciliten la presentación de información agrícola relevante, contribuyendo así a la toma de decisiones estratégicas orientadas a la obtención la soberanía alimentaria de Canarias.

Para ello, se desarrolla una aplicación full-stack que permite representar de forma visual las extensiones geográficas de los distintos terrenos y cultivos sobre un mapa interactivo. La aplicación permite la importación de múltiples conjuntos de datos así como la edición de los ya existentes, facilitando de esta manera su adaptación a nuevas necesidades. Además, incorpora panel de control sencillo e intuitivo que muestra métricas relevantes sobre los cultivos y terrenos, permitiendo obtener una visión global sobre los mismos.

En cuanto a su implementación, se ha desarrollado siguiendo una Arquitectura Hexagonal, utilizando para el desarrollo del front-end Typescript como lenguaje de programación, Vuejs como framework y Vuetify como librería de componentes. Para el back-end se utiliza Java como lenguaje de programación, Spring Boot como framework para la creación de una API REST y PostgreSQL como base de datos relacional. El despliegue de la aplicación se realiza mediante contenedores Docker, coordinados usando Docker Compose.

Palabras clave: Arquitectura Hexagonal, Front-End, Back-End, Visualizador de cultivos, Soberanía Alimentaria.

Abstract

The objective of this Final Degree Project is to develop a visualizer for planting distribution in the Canary Islands. The main objective is to provide tools that offer the presentation of relevant agricultural information, contributing to strategic decision-making aimed at achieving food sovereignty in the Canary Islands.

In order to achieve it, a full-stack application is developed that visually represents the geographical extensions of different lands and plantings on an interactive map. The application allows the import of multiple datasets as well as editing of existing ones, facilitating its adaptation to new needs. It also includes a simple and intuitive dashboard that displays relevant metrics about plantings and lands, providing a global view.

Regarding its implementation, it was developed following a Hexagonal Architecture, using Typescript as the programming language, Vuejs as the framework, and Vuetify as the component library for the front-end development. The back-end uses Java as the programming language, Spring Boot as the framework for creating a REST API, and PostgreSQL as the relational database. The application is deployed using Docker containers, coordinated using Docker Compose.

Keywords: *Hexagonal Architecture, Front-End, Back-End, Plantings visualizer, Food Sovereignty.*

Índice general

1. Introducción	1
1.1. Introducción y antecedentes	1
1.2. Objetivos y planificación	3
1.3. Estructura del documento	4
2. Estado del arte	6
2.1. Antecedentes históricos y problemas actuales	6
2.2. Sistema de Información Geográfica	6
2.3. Tecnologías desarrolladas	8
2.3.1. GeoJSON	8
2.3.2. Herramientas de navegación y visualización de mapas	10
2.4. Aplicaciones existentes	11
2.4.1. Visor de GRAFCAN	11
2.4.2. Geoportal	12
2.4.3. Sistema de Información Geográfica de datos agrarios (SIGA)	13
2.5. Perspectivas futuras y tendencias	14
3. Diseño e implementación	15
3.1. Arquitectura Hexagonal	15
3.2. Diseño e implementación de la aplicación	17
3.3. Back-end, diseño e implementación	18
3.3.1. Capa de dominio	18
3.3.2. Entidades	19
3.3.3. Objetos valor	20
3.3.4. Enumerados	20
3.3.5. Capa de aplicación	20
3.3.6. Capa de adaptadores	21
3.4. Front-end, diseño e implementación	22
3.4.1. Capa de dominio	22
3.4.2. Capa de aplicación	23
3.4.3. Capa de adaptadores	23
4. Desarrollo	25
4.1. Pruebas	25
4.1.1. Pruebas del back-end	25
4.1.2. Pruebas del front-end	26
4.2. Documentación	26
4.3. Despliegue	27
4.4. Producto final	28

4.4.1. Visualizador de distribución de terrenos y cultivos	28
4.4.2. Importador de datos	29
4.4.3. Gestión de terrenos	29
4.4.4. Gestión de cultivos	31
4.4.5. Gestión de productores	33
4.4.6. Gestión de subvenciones	35
4.4.7. Panel de control	36
5. Experimentación	38
5.1. Preparación	38
5.2. Resultados	39
6. Presupuesto	41
7. Conclusiones y líneas futuras	43
7.1. Conclusiones	43
7.2. Líneas futuras	43
8. Conclusions and Future Work	45
8.1. Conclusions	45
8.2. Future Lines	45
Bibliografía	46

Índice de figuras

1.1. Cultivos con mayor superficie en Canarias	1
1.2. Exportaciones agrarias de Canarias en periodo 2012-2023	2
1.3. Exportaciones agrícolas de Canarias en el pasado 2023	2
1.4. Importaciones agrícolas en el pasado 2023	3
1.5. Diagrama de Gantt	4
2.1. Ejemplo de capas de un SIG	7
2.2. Tipos de geometrías ofrecidas por GeoJSON	8
2.3. Ejemplo de sintaxis y representación de una colección de figuras	9
2.4. Ejemplo de sintaxis de una Feature	9
2.5. Visor de GRAFCAN donde se muestra un Mapa de Cultivos	12
2.6. Uso de Geoportal	13
2.7. Visor de SIGA donde se muestra un Mapa de Cultivos	14
3.1. Esquema de Arquitectura Hexagonal	16
3.2. Diagrama de la aplicación	17
3.3. Esquema del back-end	18
3.4. Entidades	19
3.5. Diagrama Front-end	22
4.1. Pantalla inicial de la documentación Swagger	26
4.2. Diagrama de despliegue	27
4.3. Visualizador de cultivos	28
4.4. Vista previa de detalles	29
4.5. Importador de datos	29
4.6. Listado de terrenos	30
4.7. Métricas de terrenos	30
4.8. Mapa de terrenos	31
4.9. Vista de edición de un terreno	31
4.10 Listado de cultivos	32
4.11 Mapa de cultivos	32
4.12 Vista de edición de un cultivo	33
4.13 Vista de información de un cultivo	33
4.14 Listado de productores	34
4.15 Vista de edición de un productor	35
4.16 Listado de subvenciones	36
4.17 Vista de edición de una subvención	36
4.18 Vista del panel de control	37
5.1. Ejemplo de exportación de geometrías de una isla	38

5.2. Importación de datos a la aplicación	39
5.3. Visualización de los datos	40
5.4. Visualización de métricas	40

Índice de tablas

6.1. Presupuesto	41
6.2. Presupuesto mensual de mantenimiento	42

Capítulo 1

Introducción

1.1. Introducción y antecedentes

El fenómeno de la globalización juega un papel fundamental a día de hoy en multitud de sectores de Canarias, entre los que se encuentra el agrario. Según la Revista Atlántida, en un estudio publicado en 2024, Canarias cuenta con una superficie agraria útil del 18,41 % de acuerdo a la última actualización conjunta de mapas de cultivo 2008-2015, de la cual el 30,30 % está cultivada y el 60,41 % está en desuso [8].

En la Figura 1.1 puede verse una gráfica con la distribución de los distintos tipos de cultivos en las islas tomando como referencia Diciembre de 2021, de acuerdo a la Estadística Agraria y Pesquera de Canarias 2021 [1].

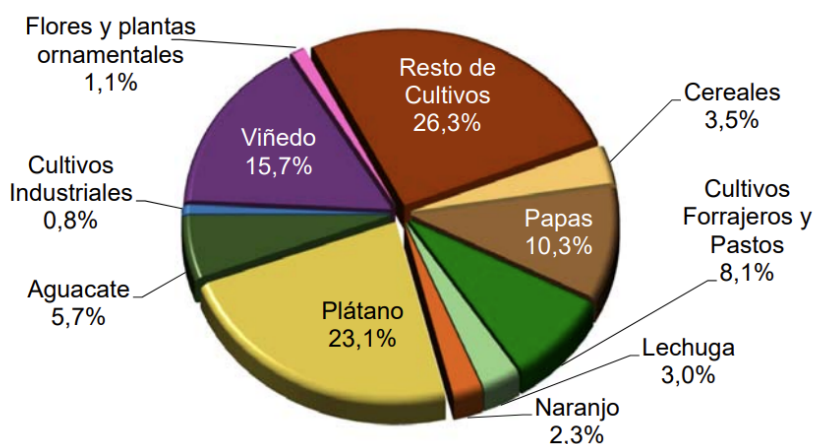


Figura 1.1: Cultivos con mayor superficie en Canarias

De acuerdo al Servicio de Estadística de la Conserjería de Agricultura, Ganadería, Pesca y Soberanía Alimentaria del Gobierno de Canarias, y tal y como se indica en la Figura 1.2, la principal exportación agraria de las islas es el plátano, con 4.185.122 toneladas exportadas en el periodo desde el año 2012 al 2023, representando un 76 % del peso de todas las exportaciones, seguido por el tomate con 632.255 toneladas, lo que equivaldría a un 11 % del peso total. El plátano supuso en dicho periodo un total de 2.272.206.000 euros en concepto de exportaciones, lo que es un 69 % del total respecto al resto de

productos agrarios, seguido de nuevo por el tomate, con un ingreso de 455.905.000 euros (un 14 %).

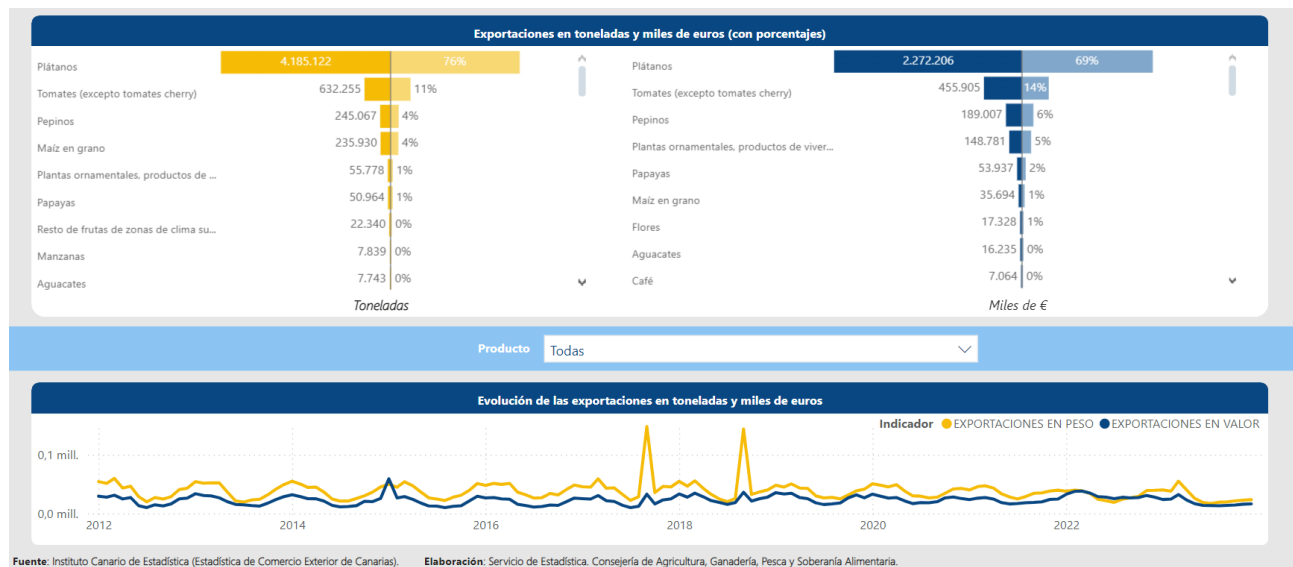


Figura 1.2: Exportaciones agrarias de Canarias en periodo 2012-2023

En cuanto al año 2023, tal y como puede verse en la Figura 1.3, la situación es bastante similar a la del conjunto de últimos años: la importancia de la exportación de plátanos es innegable con una cantidad de 297.623 toneladas, las cuales representan un 87 % respecto al total y suponen unos ingresos de 167.366.000 euros (un 76 % del total de los ingresos por exportaciones agrícolas).

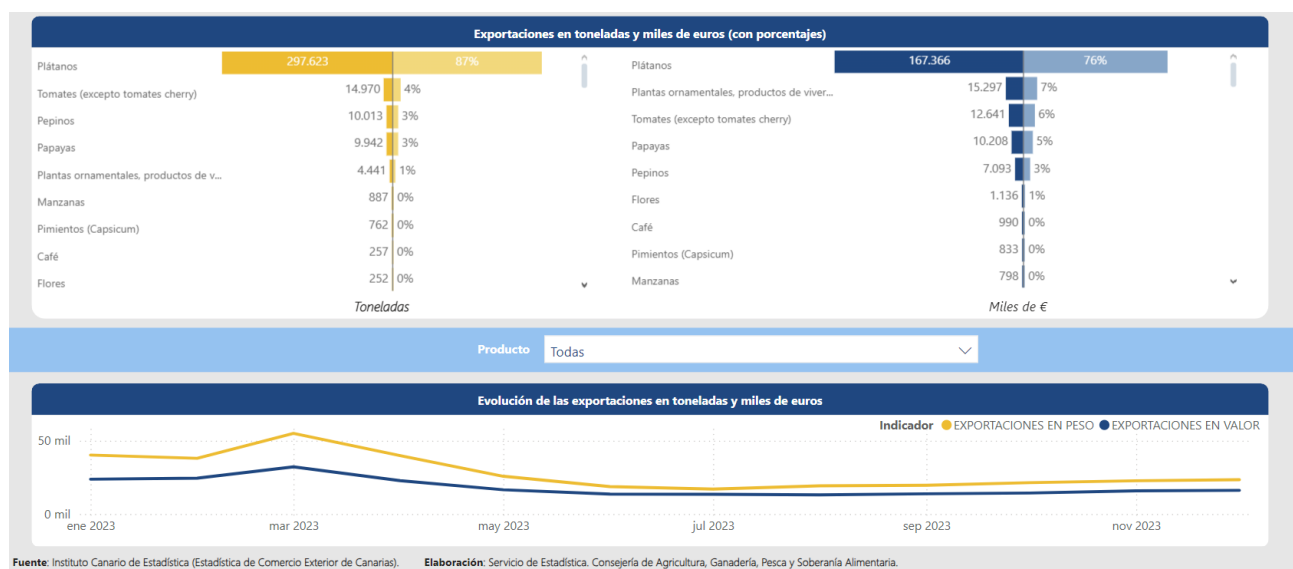


Figura 1.3: Exportaciones agrícolas de Canarias en el pasado 2023

Pasando a las importaciones del año 2023, se observa en la Figura 1.4 que las dos primeras posiciones corresponden con cereales: maíz en grano y trigo-morcajo, representando un 30 % y 19 % de las importaciones respectivamente: 21.506 toneladas de maíz en grano y 13.605 toneladas de trigo y morcajo. Ambos son seguidos por la papa (incluidas las de siembra), con 10.943 toneladas importadas, las cuales suponen un 15 % del total.

Sin embargo, en los gastos con motivo de importación se observa que es la papa la que lidera la gráfica con 7.226.000 euros, lo que supone un 13 % del total de los ingresos. Es seguida por el maíz en grano con un gasto de 5.436.000 euros (un 10 % del total) y el café. Este último no se encuentra entre los 9 productos agrícolas con más toneladas importadas, pero en cuanto a gastos de importación se encuentra en tercer lugar con 4.336.000 euros.

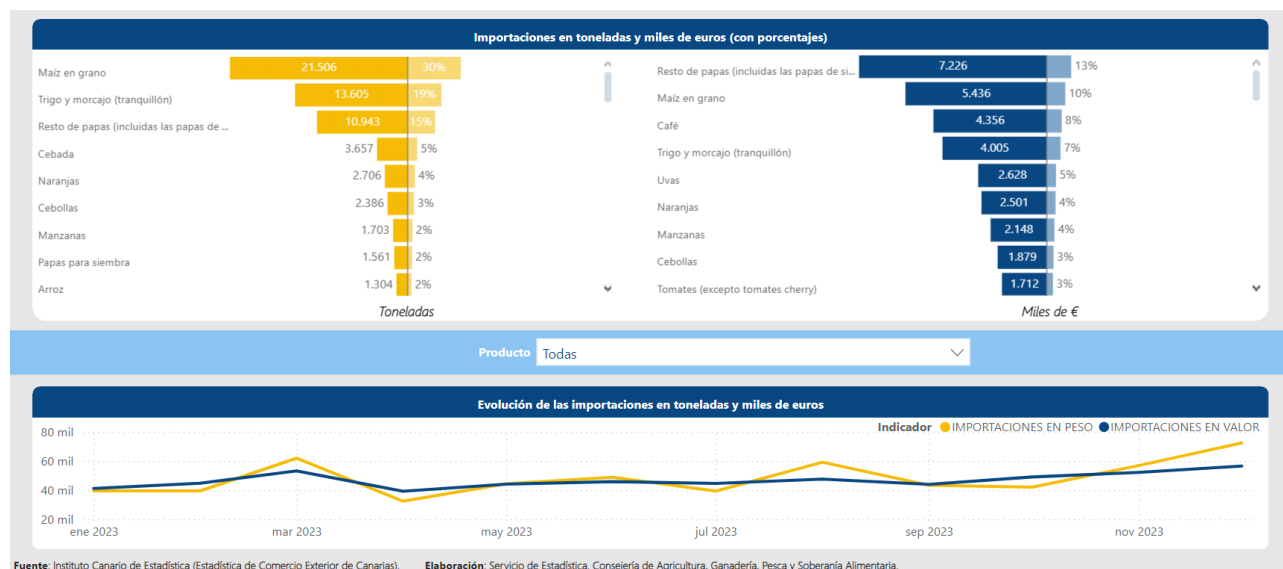


Figura 1.4: Importaciones agrícolas en el pasado 2023

1.2. Objetivos y planificación

El objetivo principal de este proyecto es desarrollar una aplicación full-stack que permita visualizar e interactuar con los terrenos de cultivo de Canarias. Esta aplicación daría la capacidad a la Consejería de Agricultura, Ganadería, Pesca y Soberanía Alimentaria del Gobierno de Canarias de tener una visión global de los cultivos de Canarias y, por lo tanto, ayudar en la toma de decisiones en pos de conseguir la soberanía alimentaria de las islas.

Los objetivos específicos del proyecto son los siguientes:

- Estudiar y trazar un plan de explotación en base a los conjuntos de datos disponibles.
- Acotar en el mapa la extensión de las distintas fincas.
- Mostrar información sobre el tipo de cultivo en producción.
- Dar la posibilidad de agrupar las fincas según distintos criterios.
- Utilizar buenas prácticas de desarrollo y diseño para conseguir una aplicación mantenible y escalable en el tiempo.

Para conseguir los objetivos anteriores, el desarrollo del proyecto se plantea de forma iterativa. El plan de trabajo incluye la realización de las siguientes tareas:

- *Tarea 1.* Documentación, investigación y análisis del estado del arte.
- *Tarea 2.* Diseño del prototipo de la aplicación web.
- *Tarea 3.* Diseño de las entidades y dominio de la aplicación.
- *Tarea 4.* Diseño y creación del back-end.
 - Implementación de la capa de dominio.
 - Creación de los servicios.
 - Creación de los controladores.
- *Tarea 5.* Diseño y creación del front-end.
 - Implementación de una vista principal.
 - Incluir mapa de las islas en la vista.
 - Graficar polígonos en el mapa.
 - Incluir filtros para el agrupamiento.
 - Crear una vista de un panel de visualización de datos.
- *Tarea 6.* Redacción de la memoria.
- *Tarea 7.* Presentación final.

La Figura 1.5 muestra un diagrama de Gantt con la planificación inicial planteada para el proyecto.



Figura 1.5: Diagrama de Gantt

1.3. Estructura del documento

El resto de la presente memoria se divide en los siguientes capítulos:

- *Capítulo 2 - Estado del arte.* Realiza una revisión bibliográfica de las herramientas y tecnologías que se han desarrollado en este ámbito, así como las aplicaciones que las utilizan.

- *Capítulo 3 - Diseño e implementación.* Detalla el diseño de la aplicación full stack realizada a nivel tecnológico, así como diversos detalles de su implementación.
- *Capítulo 4 - Desarrollo.* Muestra cada una de las funcionalidades disponibles en la aplicación. Además, explica en detalle como se realiza el testing, documentación y despliegue de la misma.
- *Capítulo 5 - Experimentación.* Prueba de funcionamiento de la aplicación a través de un caso práctico.
- *Capítulo 6 - Presupuesto.* Presupuesto para la realización del proyecto.
- *Capítulo 7 - Conclusiones y líneas futuras.* Principales conclusiones del trabajo, limitaciones del mismo y mejoras que se pueden implementar en el futuro.
- *Capítulo 8 - Conclusions and Future Work.* Conclusiones y líneas de trabajo futuras escritas en idioma inglés.

Capítulo 2

Estado del arte

2.1. Antecedentes históricos y problemas actuales

De acuerdo con lo expuesto en el Capítulo 1, el producto agrícola con mayor impacto tanto a nivel de toneladas exportadas como ingresos percibidos se trata del plátano de Canarias. Sin embargo, a veces hay excedente de producción para las necesidades del mercado, como ocurrió en el verano de 2024, con una destrucción o pica aprobada de 13 millones de kilos de plátano.¹

Por otra parte, la papa sufrió en el año 2023 una serie de problemas que impidió su correcto abastecimiento en el mercado Canario: los cultivos de este tubérculo vieron mermada su producción local en más de un 60 % a causa de, entre otros, la sequía y el calor extremo. A ello se le sumó el bloqueo de importación de papas desde Inglaterra debido a la aparición de una plaga de escarabajo colorado, lo que propició a una subida de precios dada la escasa oferta en las islas².

Estos acontecimientos ponen sobre la mesa la necesidad de disponer de herramientas que permitan ajustar la producción tanto para la exportación como para satisfacer las necesidades de las islas, intentando alcanzar de esta manera la soberanía alimentaria en el archipiélago Canario.

Es por ello que con la aplicación que se presenta en este Trabajo Final de Grado se busca ofrecer herramientas que permitan, gracias a la disposición de la información relativa a los cultivos de Canarias, intentar conseguir su soberanía alimentaria.

2.2. Sistema de Información Geográfica

Un Sistema de Información Geográfica (de ahora en adelante SIG), también conocido por sus siglas en inglés GIS (Geographical Information System), se define como el “con-

¹https://www.eldiario.es/canariasahora/eldiario-agricola/agricultura/canarias-cierra-verano-destruccion-platano-pica-13-millones-kilos_1_11654798.html

²<https://gastrocanarias.com/noticia/la-crisis-de-la-papa-en-canarias-amenaza-con-extenderse-hasta>

junto de herramientas cuyo objetivo es recolectar, almacenar, recuperar, transformar y desplegar datos espaciales del mundo real para un grupo particular de propósitos” [6].

Los SIG funcionan de forma similar a un gestor de bases de datos geográficos que van asociados a un mapa digital y pueden ser consultados. Separan la información y la almacenan como un conjunto de capas con distintas temáticas como las que pueden verse en la Figura 2.1 y que pueden ser relacionadas geográficamente. [10]

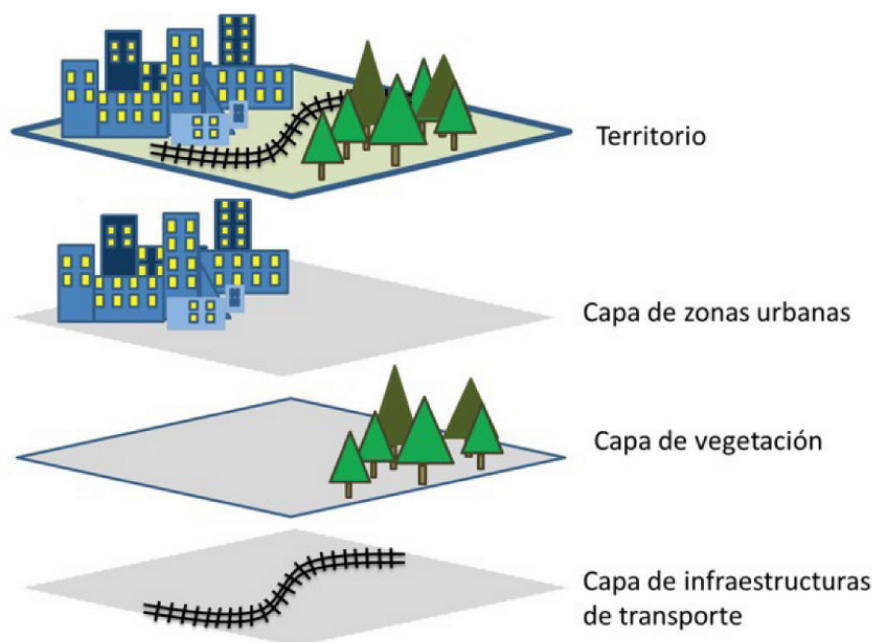


Figura 2.1: Ejemplo de capas de un SIG

Los SIG surgen en el año 1965 cuando Howard Fisher creó el Laboratorio de Gráficos por Ordenador y Análisis Espacial de Harvard con algunos programas pioneros como el ODYSSEY³.

A continuación se definen algunos ejemplos de software SIG, comenzando con CARTO⁴, que ofrece servicio en la nube de análisis y visualización de datos espaciales entre otras cosas.

Por otra parte, QGIS⁵ se trata de una alternativa libre de código abierto que permite a los usuarios crear, manipular y visualizar datos espaciales. A día de hoy se trata de una herramienta ampliamente usada y en constante desarrollo, situándose como uno de los principales softwares SIG de código abierto [5].

Otro software SIG es ArcGIS Pro⁶, software SIG de escritorio de la empresa Esri para poder visualizar, editar, analizar y compartir información geográfica. Forma parte de la plataforma ArcGIS, donde hay otros software SIG similares como ArcGIS Online y ArcGIS Enterprise.

³<https://www.esri.com/es-es/what-is-gis/history-of-gis>

⁴<https://carto.com/>

⁵<https://qgis.org/>

⁶<https://www.esri.com/es-es/arcgis/products/arcgis-pro/overview>

2.3. Tecnologías desarrolladas

A día de hoy hay multitud de tecnologías relacionadas con información geográfica, desde los Sistemas de Información Geográfica, desde formatos para la serialización de información (GeoJSON) hasta herramientas de navegación y visualización de mapas como OpenStreetMap, Google Maps o Leaflet.

A continuación se realiza una breve descripción de cada una de estas tecnologías relevantes para el proyecto.

2.3.1. GeoJSON

GeoJSON⁷ es un formato de archivos que permite codificar información geográfica haciendo uso del estándar JSON. La especificación del formato original data del año 2008 [2] y, hoy en día, GeoJSON es ampliamente utilizado.

Un ejemplo de uso lo encontramos en la librería Leaflet, que es capaz de hacer uso de GeoJSON⁸ para leer información espacial y plasmarla en sus mapas, así como convertir sus propias geometrías a GeoJSON [4]. Al ser un estándar abierto, multitud de aplicaciones SIG como CARTO⁹ o ArcGIS¹⁰, permiten su soporte, ofreciendo herramientas de importación y exportación de GeoJSON.

GeoJSON permite representar una serie de geometrías como las que pueden verse en la Figura 2.2, desde geometrías simples como *Point*, *LineString* o *Polygon*, donde solo hay un tipo de forma, pasando por las multi-parte como *MultiPoint*, *MultiLineString* y *MultiPolygon* en las que hay varias formas de un único tipo, hasta las colecciones geométricas (o *GeometryCollection*) donde hay varias formas de múltiples tipos [7].

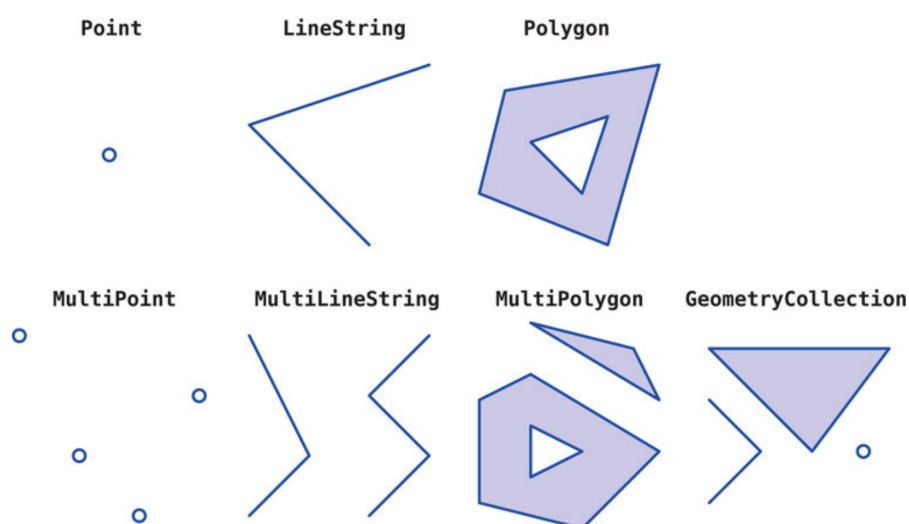


Figura 2.2: Tipos de geometrías ofrecidas por GeoJSON

⁷<https://geojson.org>

⁸<https://leafletjs.com/examples/geojson/>

⁹<https://carto.com/glossary/geojson-format>

¹⁰<https://doc.arcgis.com/en/arcgis-online/reference/geojson.htm>

En lo que se refiere a su sintaxis, tal y como puede verse en el ejemplo de una colección geométrica de la Figura 2.3, donde puede verse el conjunto de distintas formas, se basa en una serie de atributos como su tipo (haciendo referencia a los tipos de formas mencionadas anteriormente) y sus coordenadas en el plano. Además, también podemos hablar de *Features*: figuras combinadas con atributos no espaciales dentro de *properties*. Estos atributos contienen información de distinto tipo formada por pares de nombre-valor y en la Figura 2.4 puede verse la sintaxis básica de una *Feature*. Además, las *Feature*, de forma similar a las geometrías, pueden agruparse en colecciones de figuras llamadas *FeatureCollection*.

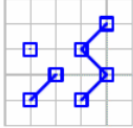
Type	Example
"GeometryCollection"	 <pre> { "type": "GeometryCollection", "geometries": [{ "type": "Point", "coordinates": [10, 30] }, { "type": "MultiLineString", "coordinates": [[[10, 10], [20, 20]], [[40, 40], [30, 30], [40, 20], [30, 10]]] }] } </pre>

Figura 2.3: Ejemplo de sintaxis y representación de una colección de figuras

```

{
  "type": "Feature",
  "geometry": {...},
  "properties": {
    "color": "red",
    "area": 3272386
  }
}

```

Figura 2.4: Ejemplo de sintaxis de una Feature

2.3.2. Herramientas de navegación y visualización de mapas

Google Maps

Google Maps¹¹ se trata de una plataforma que proporciona información geoespacial a nivel mundial. Combina imágenes satelitales con datos geoespaciales y permite también obtener vistas de los entornos mediante StreetView. Fue presentada en febrero de 2005, creada por los hermanos Lars y Jends Rasmussen: cofundadores de la empresa *Where 2 Technologies* dedicada a la creación de soluciones de mapeo, la cual fue comprada por Google en Octubre de 2004 [11].

También ofrece un servicio de API, como por ejemplo la API de Maps Javascript ¹², que se trata de una API web que permite integrar mapas interactivos en aplicaciones web, ofreciendo opciones de personalización para enriquecer la experiencia de usuario. Permite visualizar ubicaciones alrededor del mundo en 2D y 3D.

Para uso de usuario, con la interacción del mapa con un uso común, Google Maps es gratuito. Sin embargo, para uso de desarrollo y uso de sus distintas APIs conlleva un costo económico, siendo un primer número de peticiones gratuito y siendo de pago al sobrepasarse¹³.

OpenStreetMap

OpenStreetMap¹⁴ se trata de un proyecto que busca crear y proporcionar mapas de calles. Surge en la universidad la UCL (University College of London) en Julio del 2004 fundado por Steve Coast [9], y tiene como objetivo crear un conjunto de datos de mapas de libre uso y editables.

A día de hoy provee datos de mapas para miles de páginas webs, aplicaciones móviles y dispositivos, siendo sus datos de libre uso para cualquier propósito siempre y cuando se haga mención a la plataforma y sus contribuyentes¹⁵.

Los datos de OpenStreetMap son actualizados por la propia comunidad de forma continua, consiguiendo así gran exactitud incluso en situaciones donde hay cambios ya que son los propios usuarios los que lo actualizarán. Sin embargo, existe el riesgo de que el mapa pueda ser víctima de vandalismo [3].

Su uso a nivel de usuario es simple e intuitivo: ofrece una interfaz con la que se puede interactuar con el mapa y buscar la información que se necesite en el momento. Para desarrolladores, entre otras cosas, ofrece un sistema de tiles¹⁶.

¹¹<https://www.google.es/maps/>

¹²<https://developers.google.com/maps/documentation/javascript/overview?hl=es-419>

¹³<https://developers.google.com/maps/billing-and-pricing/pricing?hl=es-419>

¹⁴<https://www.openstreetmap.org>

¹⁵<https://www.openstreetmap.org/about>

¹⁶<https://operations.osmfoundation.org/policies/tiles/>

Leaflet

Leaflet¹⁷ se trata de una librería de código abierto de JavaScript que permite la inserción de mapas interactivos en aplicaciones. Originalmente creada por Volodymyr Agafonkin, hoy en día es mantenida por una gran comunidad que tiene detrás.

De base, es muy ligero, y permite trabajar con un sistema de capas, con marcadores, popups, superposición de imagen y GeoJSON entre otros. Es interactivo, con multitud de controles tanto a nivel de teclado como visuales clicables, con controles de mapa que van desde el zoom hasta la escala. Presenta opciones de personalización y es compatible con multitud de navegadores móviles y de escritorio.

Además, Leaflet tiene un potente ecosistema de plugins¹⁸ con multitud de utilidades de distinto tipo, entre los que para este trabajo destaca Geoman, una potente herramienta¹⁹ en forma de plugin que permite dibujar, editar y trabajar sobre capas de geometrías, encargándose de la lógica del manejo de geometrías en las distintas capas y ofreciendo una herramienta gráfica intuitiva para trabajar con estas.

2.4. Aplicaciones existentes

En la actualidad existen múltiples aplicaciones que permiten visualizar información relativa a los mapas de cultivo del archipiélago canario. A continuación se presentan aquellas más relevantes dentro del ámbito de Canarias.

2.4.1. Visor de GRAFCAN

El visor de GRAFCAN²⁰ es una herramienta que ofrece Cartográfica de Canarias S.A. (GRAFCAN)²¹ que permite visualizar multitud de datos cartográficos del archipiélago en forma de mapa interactivo.

Presenta un mapa de las Islas Canarias interactivo en el cual se puede seleccionar entre un gran número de capas disponibles, representando cada una de ellas distintos conjuntos de datos cartográficos agrupados, entre los que se encuentran, por ejemplo, cartografías básicas, turismo y equipamientos, caracterización del suelo, etc. Todo ello con su correspondiente leyenda y posibilidad de acceso a los datos en crudo.

En el caso que interesa para el presente trabajo, se encuentra el conjunto de datos de Agricultura, Ganadería y Pesca, dentro del cual se encuentra el Mapa de Cultivos. Dicho mapa es interactivo proporciona información relevante como el tipo de cultivo en cada zona así como la fecha de recogida de los datos.

¹⁷<https://leafletjs.com>

¹⁸<https://leafletjs.com/plugins.html>

¹⁹<https://geoman.io/>

²⁰<https://visor.grafcan.es/visorweb/>

²¹<https://https://www.grafcan.es>

Esta plataforma, tal y como puede verse en la Figura 2.5, ofrece a su vez un conjunto de herramientas como zoom, cambios de vistas, creación de enlaces a vistas, dibujos sobre el mapa, medición de distancias, cálculo de rutas, impresión de mapas, vista 3D y exportaciones de mapas entre otras.

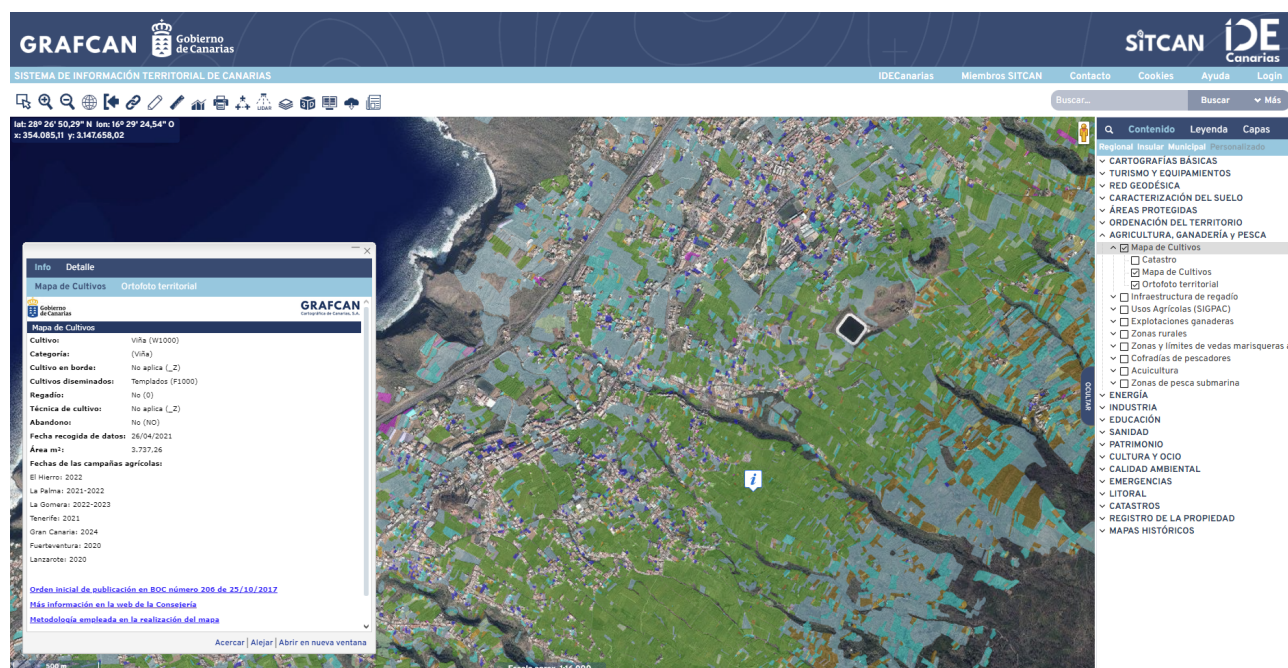


Figura 2.5: Visor de GRAFCAN donde se muestra un Mapa de Cultivos

2.4.2. Geoportal

Geoportal²² es una plataforma web ofrecida por el Ministerio de Agricultura, Pesca y Alimentación del Gobierno de España que dispone cartografía de los distintos territorios de España. Cuenta con una serie de mapas base de distintas temáticas sobre los cuales es posible seleccionar distintos conjuntos de datos de diferentes índoles, como agricultura, alimentación, naturaleza, costas, ganadería y pesca entre otros.²³

Tal como puede verse en la Figura 2.6, la aplicación dispone de una serie de herramientas, como por ejemplo, aquellas dedicadas a la medición o localización por distintos criterios como dirección, topónimo o código postal.

En relación a la agricultura, ofrece distintos mapas de cultivo asociados a los periodos de 1980-1990 y 2000-2010. Una vez se accede a dichos recursos, es posible hacer click en las distintas áreas para obtener información sobre el uso del terreno, la extensión en metros cuadrados, hectáreas, la sobrecarga entre otros.

²²<https://sig.mapama.gob.es/geoportal/>

²³<https://www.mapa.gob.es/es/cartografia-y-sig/ide/geoportal/>

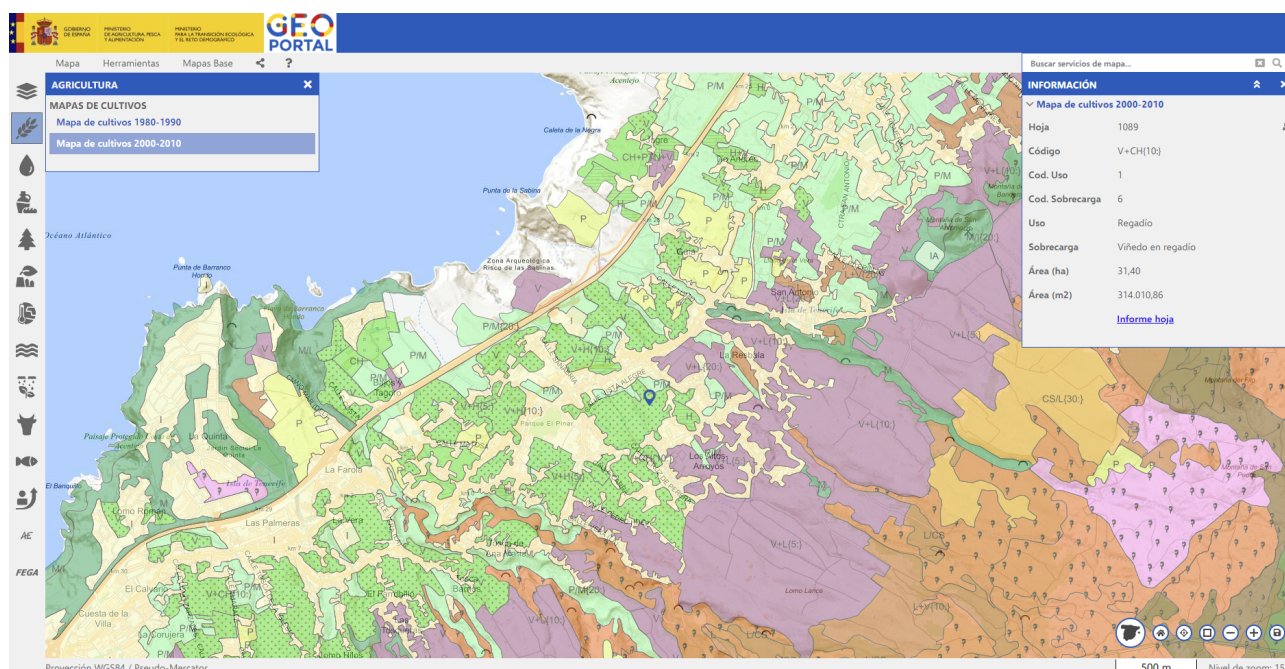


Figura 2.6: Uso de Geoportal

2.4.3. Sistema de Información Geográfica de datos agrarios (SIGA)

El Sistema de Información Geográfica de Datos Agrarios (SIGA)²⁴ es una herramienta web desarrollada por el Ministerio de Agricultura, Pesca y Alimentación del Gobierno de España que permite visualizar información geográfica relacionada con el sector agrario en forma de mapa a nivel estatal, entre la que se puede encontrar cartografía general, mapas temáticos sobre variables agro-climáticas, informes sobre municipios y estaciones meteorológicas, mapas de cultivos y aprovechamientos, y más.

El visor, el cual puede verse en la Figura 2.7, ofrece una serie de herramientas tales como la localización, permitiendo ubicar comunidades autónomas o municipios, un árbol de servicios en el que poder seleccionar los distintos conjuntos de datos geográficos a visualizar, una tabla de contenidos con la información sobre los conjuntos de datos cargados y sus leyendas, etc.

²⁴<https://sig.mapama.gob.es/siga/>

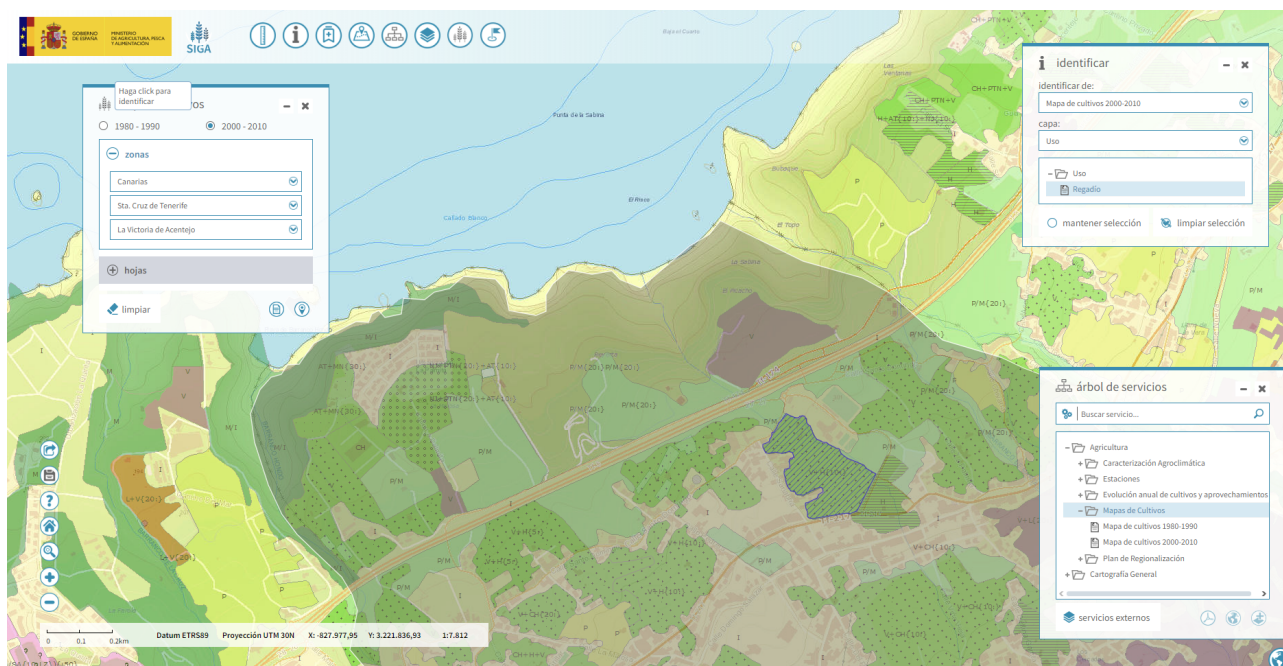


Figura 2.7: Visor de SIGA donde se muestra un Mapa de Cultivos

2.5. Perspectivas futuras y tendencias

La revolución que supone la inteligencia artificial ha tenido incidencia en cada vez más ámbitos, entre ellos el campo de la agricultura. Las tecnologías aplicadas a la visualización de datos de cultivos tenderán a sistemas automatizados, combinando imágenes satelitales, sensorización, inteligencia artificial y análisis predictivo.

La recopilación de datos en tiempo real sobre características de las zonas de cultivo como la humedad o calidad de suelo y datos climáticos junto con el uso de herramientas de inteligencia artificial pueden analizar características de las distintas zonas así como elegir cultivos adecuados para las mismas.

Los datos mencionados anteriormente, junto con los datos de consumo de productos agrarios en el archipiélago y las exportaciones, pueden generar modelos que sean capaces de realizar una aproximación las necesidades futuras de consumo de productos agrícolas en las islas, permitiendo anticipar escenarios futuros y permitiendo apoyar la toma de decisiones estratégicas, por ejemplo, en forma subvenciones y/o beneficios fiscales de tal forma que se incentive el cultivo de determinados productos en determinados momentos y/o en determinadas zonas.

Capítulo 3

Diseño e implementación

En este capítulo explora en detalle las tecnologías utilizadas en el desarrollo de la aplicación, así como su arquitectura a nivel de software y su composición. Estos aspectos son fundamentales para comprender la estructura y el funcionamiento de la aplicación.

3.1. Arquitectura Hexagonal

La arquitectura usada en esta aplicación se trata de la Arquitectura Hexagonal, también conocida como arquitectura de puertos y adaptadores. En el año 2005, Alistair Cockburn, su creador, en "The Pattern: Ports and Adapters (Object Structural)"¹, propone esta arquitectura.

De acuerdo con [12], una de las ideas principales de esta arquitectura se basa en separar el código de negocio del código tecnológico. Asegura que el lado tecnológico dependa del de negocio, lo que permite que este evolucione de forma agnóstica a la tecnología que se usa. Por otra parte, dispone de herramientas que permiten que el código tecnológico puede cambiar sin afectar al código de negocio.

La Arquitectura Hexagonal se compone principalmente de 3 capas, anillos o hexágonos, cada uno con su rol correspondiente para lograr los objetivos descritos previamente. En la Figura 3.1 puede verse un esquema visual de cómo se organizan dichas capas y componentes de las mismas que se exponen a continuación. Estas capas son las siguientes:

¹<https://alistair.cockburn.us/hexagonal-architecture>

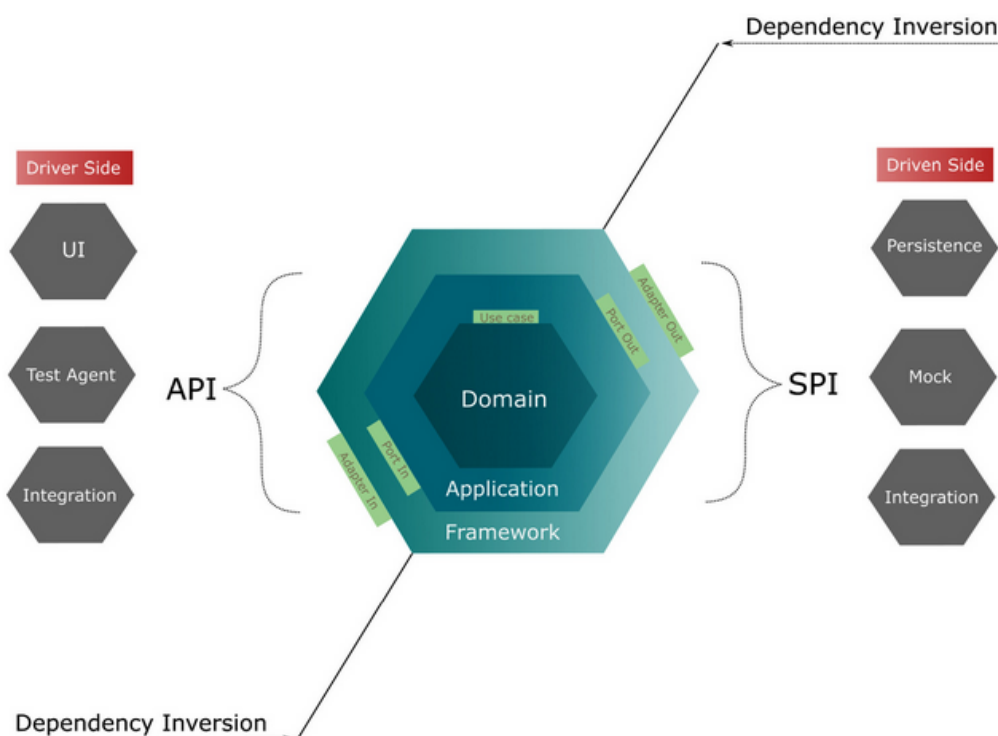


Figura 3.1: Esquema de Arquitectura Hexagonal

- **Dominio:** Se encarga de modelar el problema del mundo real dentro del software, reúne los elementos que describen los problemas centrales que el software resuelve. Está formada por entidades y objetos valor que encapsulan los datos y las reglas de negocio, manteniéndose aislados de la tecnología. Las entidades representan elementos con identidad propia dentro del dominio, y los objetos valor son componentes inmutables que se usan para componer las entidades.
- **Aplicación:** Orquesta las tareas específicas de la aplicación pero desde un punto de vista abstracto, describiendo las funcionalidades del software basadas en las reglas de negocio definidas en el dominio. Esto lo hace por medio de los casos de uso, puertos de entrada y puertos de salida.
 - *Casos de uso:* Definen el comportamiento del sistema, es decir, lo que el software ha de hacer para alcanzar un objetivo de negocio, pero sin llegar a definir detalles técnicos.
 - *Puertos de entrada:* Los puertos de entrada, también conocidos como servicios, se encargan de implementar las acciones dictadas por los casos de uso. Son estos los que llevan a cabo, de la mano de los adaptadores, las acciones para alcanzar los distintos objetivos.
 - *Puertos de salida:* Se tratan de interfaces que definen qué datos necesita el sistema desde el exterior, sin llegar a definir cómo.
- **Adaptador:** Es responsable de manejar las comunicaciones con tecnologías externas, haciendo de puente con el mundo exterior. Define cómo se exponen las funcionalidades de la aplicación y cómo se consumen datos de fuentes externas. Esto se consigue por medio de los adaptadores de entrada y adaptadores de salida.

- *Adaptadores de entrada:* Reciben solicitudes desde el exterior y usan los casos de uso para hacer las operaciones que correspondan.
- *Adaptadores de salida:* Implementan la definición de los puertos de salida y se encargan de obtener o enviar información al exterior.

La utilización de la Arquitectura Hexagonal proporciona beneficios tales como que el software está más preparado para afrontar cambios de tecnologías gracias a los puertos y adaptadores, pudiendo adoptar nuevas tecnologías haciendo cambios en estos manteniéndose la lógica de negocio aislada tal y como se comentó en la capa de dominio. También ofrece ventajas respecto la realización de pruebas en el código, ya que da flexibilidad para probar las partes críticas del código aún sin las tecnologías externas presentes. Además, tiene una estructura clara y modular que evita acoplamientos innecesarios, facilitando la evolución del sistema y ahorrando deuda técnica a largo plazo.

3.2. Diseño e implementación de la aplicación

La arquitectura de esta aplicación, tal como puede verse en la Figura 3.2, está compuesta de tres grandes bloques: front-end, back-end y base de datos. Estos componentes han sido orquestados mediante el uso de Docker Compose, de tal forma que se facilita la gestión y el despliegue de los distintos servicios en sus respectivos contenedores Docker.

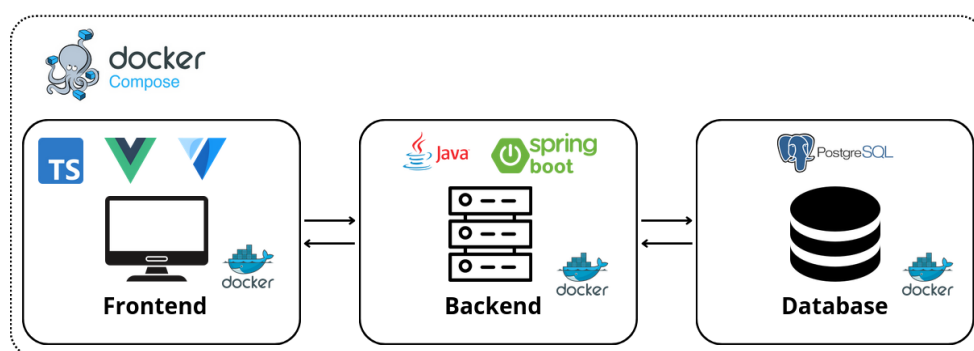


Figura 3.2: Diagrama de la aplicación

3.3. Back-end, diseño e implementación

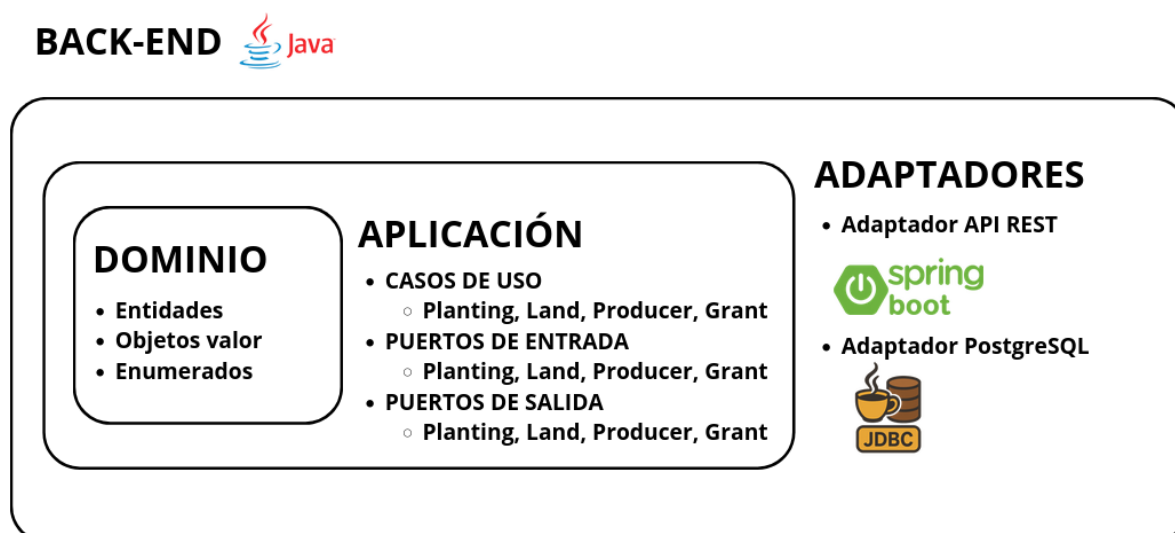


Figura 3.3: Esquema del back-end

El back-end se trata de la parte de la aplicación encargada de gestionar la lógica de negocio, procesar los datos y de interactuar para recuperar y almacenar información en la base de datos. Actúa como intermediario entre el front-end y la base de datos.

Ha sido construido utilizando Java como lenguaje de programación, utilizando Spring Boot como framework para poder construir la API REST.

3.3.1. Capa de dominio

La capa de dominio representa el modelo y las reglas del negocio, abstraída de tecnologías externas a través de las entidades, objetos valor y enumerados. En esta aplicación se definen una serie de entidades tales como las que podemos ver en el diagrama de la Figura 3.4, las cuales se presentan a continuación:

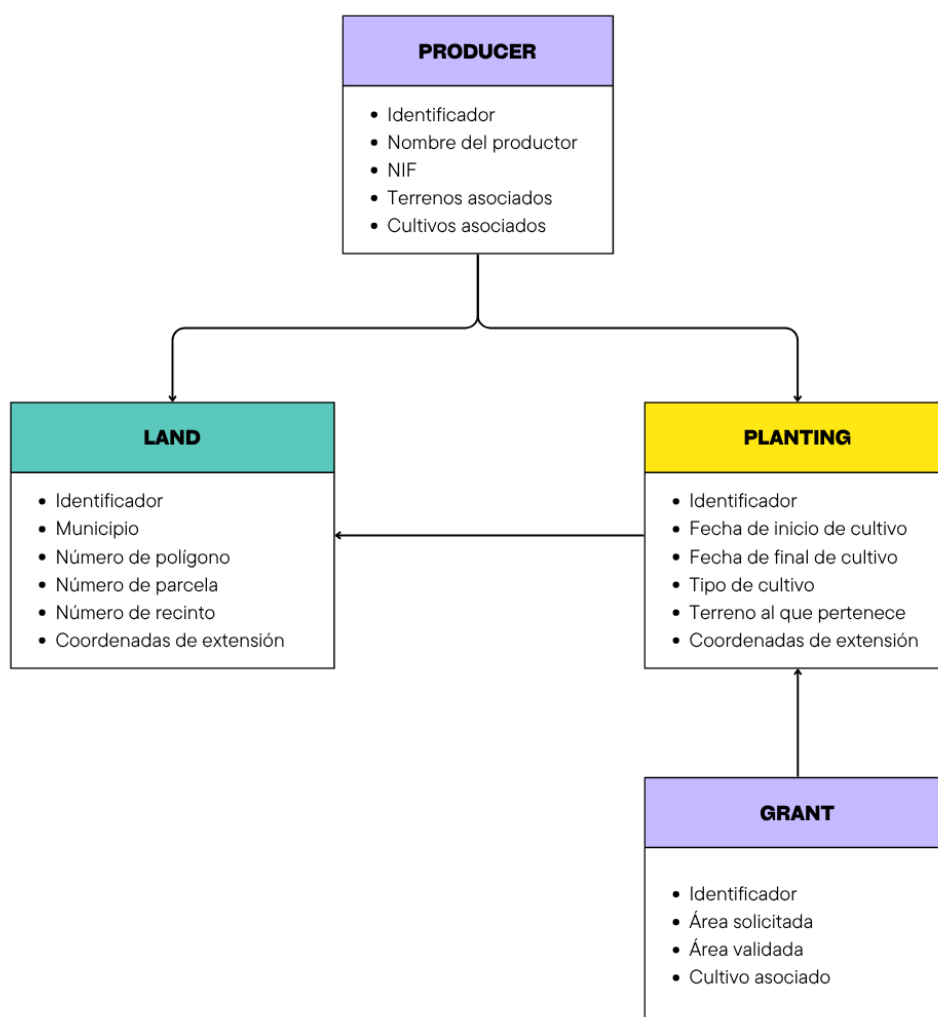


Figura 3.4: Entidades

3.3.2. Entidades

- **Planting** se trata de, junto con los terrenos, la entidad más importante del dominio, la cual representa un cultivo con características como su fecha de inicio y final de cultivo, el tipo de cultivo que tiene (papas, tomates, plátanos, etc.), la extensión geográfica almacenada en GeoJSON de dicho cultivo y, el terreno al que pertenece. Como puede verse, esta entidad se trata la encargada de almacenar la información más relevante a mostrar en el mapa.
- **Land** es la encargada de almacenar también información muy relevante para la visualización, representa los terrenos que contienen los cultivos. Almacena información sobre el municipio al que pertenece el terreno, los identificadores de polígono, parcela y recinto asociados. También posee información en formato GeoJSON.
- La entidad **Grant** es una representación de una subvención asociada a un cultivo, con información sobre el área solicitada para la misma, el área validada y el cultivo asociado a dicha ayuda.
- Finalmente, **Producer** representa bien un productor o una cooperativa, con características como nombre, número fiscal, y una serie de terrenos y cultivos asociados al

mismo.

3.3.3. Objetos valor

Un objeto valor se trata de un componente inmutable que se utiliza para la creación de las entidades. Para las características de las entidades descritas previamente se definen los objetos valor que se exponen a continuación:

- **Municipality** representa un municipio, almacenando su nombre.
- **Polygon** es un identificador numérico entero positivo asociado a un polígono.
- **Plot** es un identificador numérico entero positivo asociado a una parcela.
- **Enclosure** es un identificador numérico entero positivo asociado a un recinto.
- **NIF** almacena el número de identificador fiscal de un productor (ya sea persona física o empresa).
- **ProducerName** contiene el nombre de un productor.
- **RequestedArea** contiene el área solicitada para una subvención.
- **ValidatedArea** se trata del área validada para una subvención.

3.3.4. Enumerados

Los enumerados permiten definir una serie de valores fijos con sentido dentro del dominio. Gracias a esto se pueden delimitar valores determinados y reducir la posibilidad de errores en el código. En este proyecto se usa el siguiente enumerado:

- **ProductName** se encarga de definir los distintos tipos de cultivo que pueden presentarse, tales como tomates, plátanos, papas, etc.

3.3.5. Capa de aplicación

En la capa de aplicación se encuentran tanto los casos de uso como los puertos, descritos a continuación:

- *Casos de uso*: Se tratan de aquellos que permiten realizar operaciones CRUD (Create, Read, Update, Delete) con las distintas entidades del dominio.
- *Puertos*: En el caso de esta aplicación, son los que generan las entidades y, con ayuda de el adaptador PostgreSQL que se explicará más adelante, consiguen la persistencia de las mismas. En esta aplicación se definen la interfaces de los repositorios con acciones de creación, modificación, obtención y eliminación de entidades.

3.3.6. Capa de adaptadores

Los adaptadores se encargan de conectar la aplicación con el mundo exterior. Como adaptador de salida está el adaptador PostgreSQL, que implementa el repositorio definido en la capa de aplicación. Por otra parte, como adaptador de entrada, está el adaptador API REST, que hace uso de los casos de uso definidos en la capa de aplicación.

Adaptador API REST

Para desarrollar el adaptador de API REST² se hace uso de Spring Boot, un framework de Java basado en Spring para crear aplicaciones. El adaptador REST se encarga de ofrecer una forma de conseguir la persistencia de las entidades del dominio, exponiendo los distintos casos de uso de la aplicación a través de una interfaz HTTP. Es por medio de esta interfaz que el cliente del front-end puede interactuar con el back-end de forma sencilla y desacoplada mediante peticiones HTTP.

Adaptador PostgreSQL

PostgreSQL³ se trata de un sistema de base de datos relacional de código abierto muy potente y robusto, con más de 35 años a sus espaldas de desarrollo activo. Se sitúa como una de las soluciones más fiables en el mundo de las bases de datos y es ampliamente utilizado por multitud de desarrolladores y empresas por todo el mundo.

Por otra parte, JDBC (Java Database Connectivity)⁴ es una API diseñada para Java que permite interactuar con una base de datos, pudiendo así realizar consultas y actualizar la información de la misma. Su uso está principalmente enfocado a ser utilizada en bases de datos relacionales.

En este proyecto, para la implementación del adaptador PostgreSQL, por una parte, mediante el uso de JDBC se ha creado una clase encargada de establecer conexión con la base de datos PostgreSQL mediante el patrón Singleton⁵ de tal forma que la instancia de la conexión pueda ser reutilizada y esté unificada.

Por otra parte, se ha definido un repositorio PostgreSQL en el que se implementan las interfaces de los repositorios con las diferentes operaciones de inserción, modificación y borrado de entidades y, mediante el uso de JDBC y la definición de las sentencias SQL, se realizan las operaciones DML deseadas.

²<https://spring.io/guides/tutorials/rest>

³<https://www.postgresql.org/>

⁴<https://jdbc.postgresql.org/documentation/>

⁵<https://www.geeksforgeeks.org/jdbc-using-model-object-and-singleton-class/>

3.4. Front-end, diseño e implementación

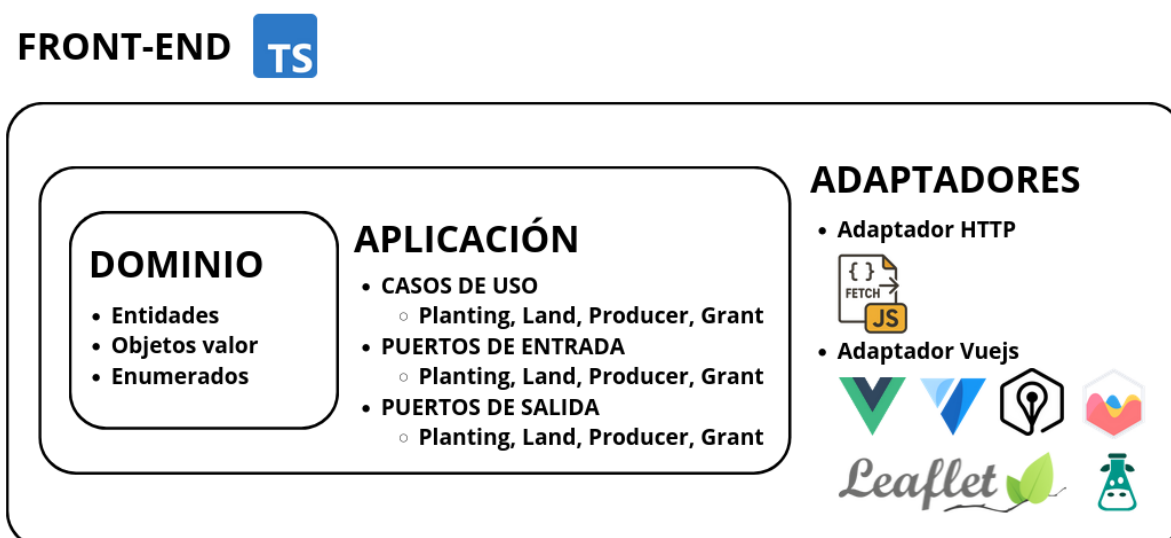


Figura 3.5: Diagrama Front-end

El front-end se trata de la capa de la aplicación con la que los usuarios interactúan directamente a través de un navegador web, y su función principal se trata de presentar los datos y permitir a los usuarios que interactúen con ellos a través de la interfaz de usuario implementada. En esta aplicación, se construye el front-end mediante el uso de las siguientes tecnologías:

Como lenguaje de programación se usa Typescript⁶, un lenguaje de programación fuertemente tipado que se basa en Javascript y al ser tipado ayuda a mejorar la robustez del código y detectar errores con mayor facilidad en tiempo de compilación. Como framework se usa Vuejs⁷, un moderno framework de Javascript para la creación de interfaces web y se usa junto a Vuetify⁸, una librería de componentes para Vuejs basada en Material Design. Para los mapas se utiliza Leaflet como librería de mapas para disponer la información geográfica de forma visual y Geoman para permitir editar formas polígonos sobre el mapa.

3.4.1. Capa de dominio

En cuanto al dominio del front-end, se trabaja con las mismas entidades que en el back-end, de tal forma que han sido traducidas al lenguaje. Esto es así dado que en ambos extremos de la aplicación, los distintos elementos del dominio tales como entidades, objetos valor y enumerados que se necesitan son idénticos.

⁶<https://www.typescriptlang.org/>

⁷<https://vuejs.org/>

⁸<https://vuetifyjs.com/en/>

3.4.2. Capa de aplicación

En la capa de aplicación se encuentran tanto los casos de uso como los puertos. En cuanto a los casos de uso, de forma similar al back-end, se definen para operaciones de creación, edición, obtención y eliminación de entidades y, en los puertos de entrada, los servicios, se implementan dichos casos de uso. Por otra parte en los puertos de salida, se definen las interfaces de los repositorios para obtener desde el exterior dichas entidades.

3.4.3. Capa de adaptadores

En la capa de adaptadores se definen las conexiones con el exterior que tendrá el front-end. Se presentan los siguientes:

Adaptador HTTP

Este adaptador implementa el puerto de salida definido en la capa de aplicación, actuando como responsable de la gestión de la comunicación entre el front-end y el back-end mediante llamadas HTTP a las rutas expuestas de la API REST del back-end.

Así puede acceder a los casos de uso del back-end previamente definidos describiendo en las peticiones la acción a realizar y las entidades involucradas.

Adaptador Vuejs

Este adaptador se encarga de trabajar con la interfaz web del front-end, dando una interfaz visual con la que poder interactuar con el sistema, haciendo una separación de distintos módulos con el objetivo de organizarlos y mantenerlos independientes.

- El **módulo de vistas** contiene las distintas pantallas de la aplicación, desde la página de inicio con el mapa interactivo hasta las distintas páginas de consulta y edición de cultivos y terrenos entre otras. Las vistas contienen componentes y orquestan la lógica para trabajar con ellos.
- El **módulo de componentes** contiene aquellos componentes hechos en Vuejs que han sido creados para satisfacer necesidades de la aplicación, como por ejemplo implementaciones de mapas, filtros o gráficas. Obtienen su información por medio de las vistas, que son las que les pasan la información con la que van a trabajar.
- El **módulo de enrutamiento** usa Vue Router para enrutar las distintas vistas de la aplicación, haciéndolas accesibles a través de distintas URL. Es también encargado de obtener la información relevante para las vistas como identificadores dentro de la propia URL para que estas tengan acceso a ella.

- El **módulo de internacionalización** `i18next`⁹ se trata de una librería que permite la internacionalización de aplicaciones al traducirlas a múltiples idiomas y está disponible para multitud de entornos, entre los que se encuentra `Vuejs`¹⁰. En esta aplicación se usa en el front-end para traducir las distintas vistas al inglés y al español por medio de su configuración en ficheros `.json` con sus traducciones.

⁹<https://www.i18next.com/>

¹⁰<https://www.locize.com/blog/i18next-vue>

Capítulo 4

Desarrollo

4.1. Pruebas

4.1.1. Pruebas del back-end

Las pruebas del back-end se han hecho utilizando las herramientas JUnit¹, un framework de pruebas unitarias para Java que permiten ejecutar pruebas automatizadas a fragmentos de código de forma aislada. También se ha utilizado MockMVC², una herramienta que da soporte a las pruebas de aplicaciones web creadas con Spring MVC. Finalmente, se ha utilizado Mockito³, un framework para simulación de objetos en pruebas de Java. En el back-end se han hecho tanto pruebas unitarias como de integración:

Las **pruebas unitarias** permiten ejecutar pruebas sobre distintos componentes del software de forma que puedan ser probados de forma aislada y controlada, sin depender de otros componentes del programa. En este caso, se hicieron usando JUnit, principalmente en pruebas sobre los objetos valor del dominio, sometiéndolos a distintos tipos de situaciones tanto donde deberían funcionar, como donde deberían fallar, siendo un total de 68 test unitarios.

Las **pruebas de integración** se han hecho con el objetivo de comprobar el funcionamiento de los endpoint CRUD que expone la API del back-end. Para ello, se utiliza MockMVC para poder probar los endpoints sin necesidad de arrancar el servidor real, y, para poder utilizar datos simulados (de ahora en adelante mocks) se utilizó Mockito, simulando respuestas de los casos de uso. Se realizaron pruebas a los distintos controladores de las entidades, siendo estas un total de 16 pruebas de integración.

¹<https://junit.org>

²<https://docs.spring.io/spring-framework/reference/testing/mockmvc.html>

³<https://site.mockito.org>

4.1.2. Pruebas del front-end

Para poder verificar el correcto funcionamiento de la interfaz de usuario del front-end, se han hecho pruebas automatizadas end-to-end usando la herramienta Cypress. Se ha diseñado un conjunto de pruebas automáticas en las principales vistas de la aplicación como, por ejemplo, el visualizador de cultivos y las distintas vistas de listado de entidades. Se han hecho un total de 54 pruebas end-to-end.

Por ejemplo, en las pruebas se han probado las vistas comprobando su correcta carga, probando direccionamientos correctos al pulsar botones que llevan a otras vistas y la presencia y el funcionamiento de sus componentes entre otras cosas.

4.2. Documentación

La documentación de la API se ha hecho mediante el uso de la herramienta Swagger, una potente herramienta open-source que usa para visualizar e interactuar con servicios REST. En este caso, tal y como puede verse en la Figura 4.1, se utiliza para generar una interfaz web de forma que se listen y enumeren los distintos endpoints de la API, correspondientes a los que se han definido en los controladores. Se muestra información como el tipo de petición que recibe cada endpoint e instrucciones sobre sus usos, tal y como puede verse en la figura. También da la posibilidad de hacer interactuar, como se mencionó previamente, con la API. Gracias a esto, se tienen de una forma muy visual y accesible, las instrucciones y requisitos para el uso de la API.

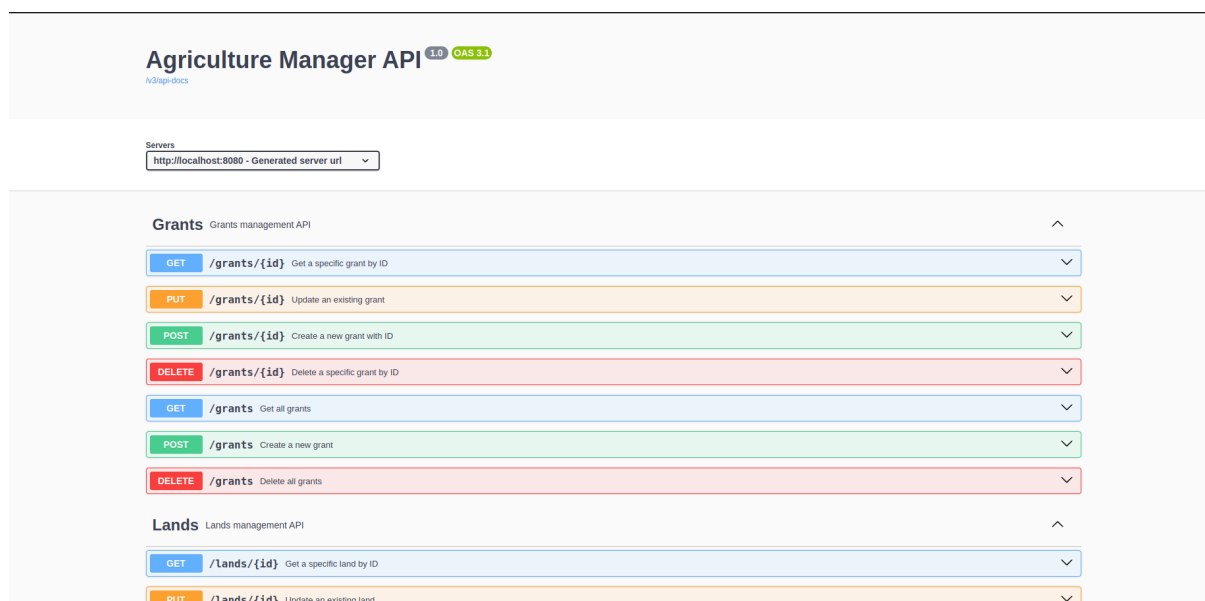


Figura 4.1: Pantalla inicial de la documentación Swagger

4.3. Despliegue

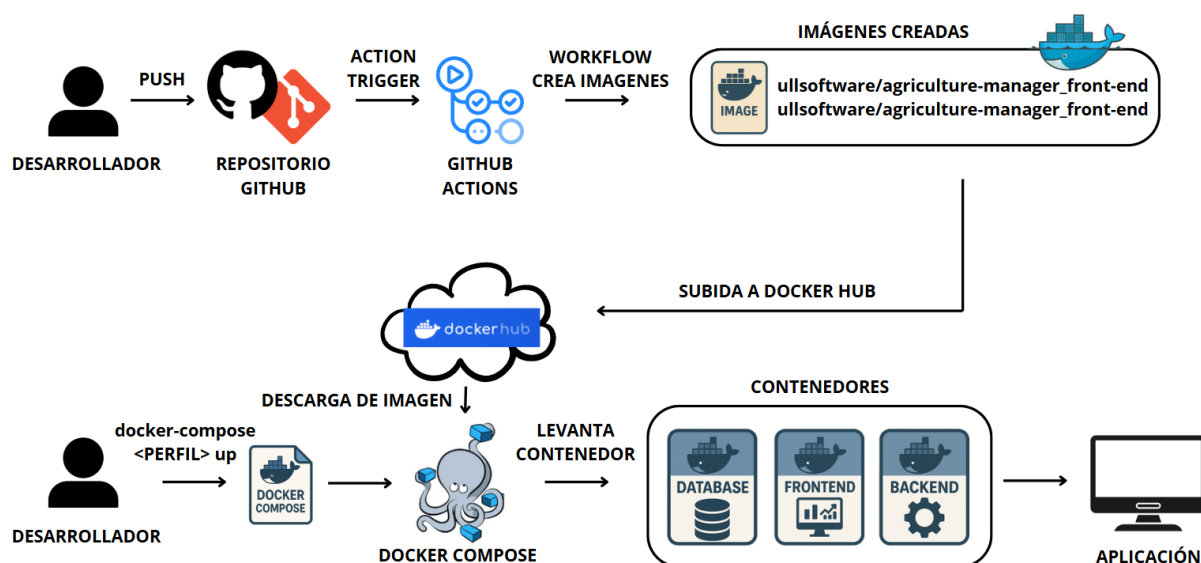


Figura 4.2: Diagrama de despliegue

Tal y como puede verse en la Figura 4.2, cuando se hace un push con cambios en el código, en función de si realiza cambios en el back-end y/o en el front-end, se dispara una Github Action. Esta action se encarga de crear la imagen correspondiente mediante los Dockerfile para posteriormente subirla a Docker Hub, el repositorio donde se aloja para que posteriormente pueda ser descargada y usada por quien lo requiera.

El despliegue se realiza a través de Docker Compose, que permite orquestar aplicaciones que usen múltiples contenedores con distintos servicios a través de las instrucciones dadas en el fichero de configuración `docker-compose.yml`, en el que se han definido una serie de perfiles en función de qué partes de la aplicación se quiera levantar en un momento dado.

Para desplegar la aplicación completa, desde el terminal se utiliza el perfil *full* en `docker-compose <perfil> up` y se descargan las imágenes de los contenedores (si no han sido descargado previamente o no hubiera habido cambios en las mismas) y se levantan los distintos contenedores.

En esta aplicación se define un contenedor **front-end** que contiene el propio front-end, otro **back-end** que contiene el back-end y un último, este externo al desarrollo, *postgresql*, que contiene una imagen de PostGIS⁴, una imagen que permite correr una base relacional de PostgreSQL con PostGIS⁵ instalado, una extensión de PostgreSQL para el manejo de información geoespacial.

Una vez levantados dichos contenedores, puede hacerse uso de los servicios accediendo a las direcciones definidas en sus configuraciones.

⁴<https://hub.docker.com/r/postgis/postgis>

⁵<https://postgis.net>

4.4. Producto final

En esta sección se muestra el funcionamiento de la aplicación del proyecto, detallando las funcionalidades que componen el producto desarrollado. A continuación se describen las distintas partes que lo conforman.

4.4.1. Visualizador de distribución de terrenos y cultivos

Se trata de la pantalla principal de la aplicación (Figura 4.3). En ella hay un mapa de Canarias que contiene polígonos que representan las áreas que ocupan los distintos terrenos, en rojo, y cultivos, en azul, tal como indica la leyenda.

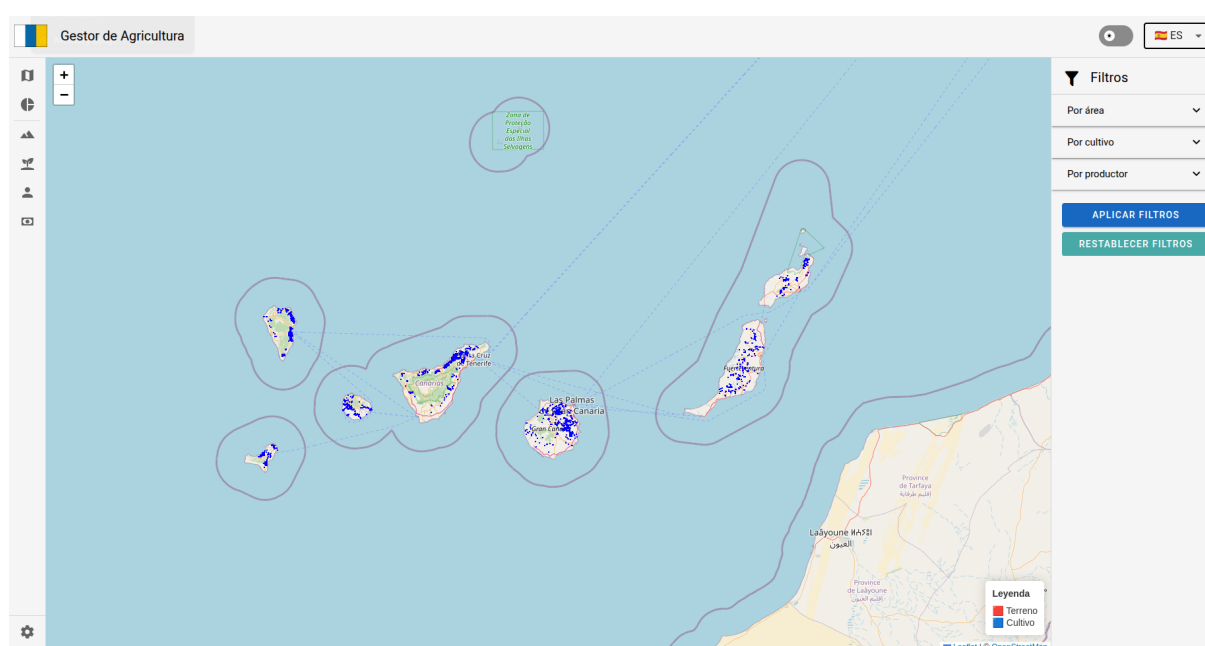


Figura 4.3: Visualizador de cultivos

El mapa es interactivo, permitiendo ajustar el zoom y moverse por el mismo y observar las áreas más de cerca, pudiendo hacer click sobre las mismas para que un resumen de su información (ya sea un terreno o un cultivo) aparezca en forma de pop-up, tal y como puede verse en la Figura 4.4. También hay una serie de filtros a la derecha de la vista para poder filtrar solo aquellos cultivos y terrenos que cumplan con restricciones como las siguientes:

- **Por área.** Isla, municipio/s.
- **Por cultivo.** Tipo de cultivo, fechas.
- **Por productor.** Nombre, NIF.

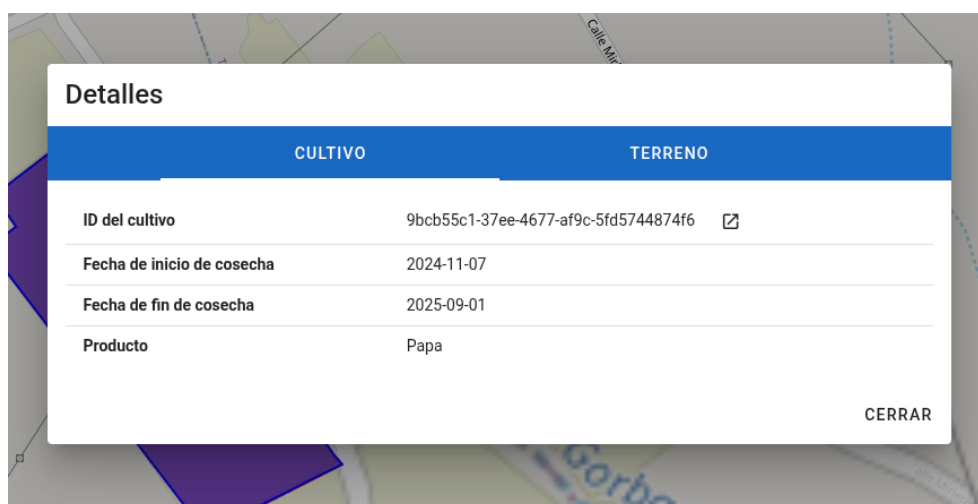


Figura 4.4: Vista previa de detalles

4.4.2. Importador de datos

Para la importación de los datos la aplicación cuenta con un importador de datos que permite, mediante la subida de un fichero JSON que contenga la descripción del conjunto de datos, transformar, crear y persistir las entidades que correspondan. También se ofrece una opción para eliminar los conjuntos de datos actuales y poder partir desde cero. En la Figura 4.5 se muestra el importador.

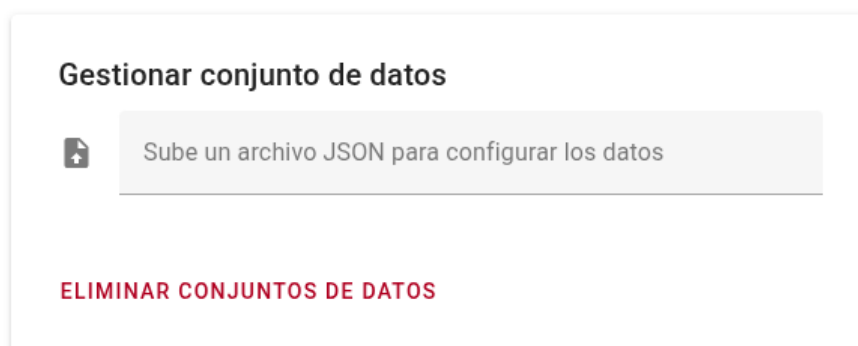


Figura 4.5: Importador de datos

4.4.3. Gestión de terrenos

Una ventaja que ofrece esta aplicación es que, además de poder visualizar los datos, permite modificar, eliminar y añadir registros, de tal forma que los datos de la aplicación puedan ser ajustados. A continuación se muestra cómo se dispone el conjunto de los datos de terrenos y las opciones para sus modificaciones.

Listado de terrenos

Tal y como se muestra en la Figura 4.6, en esta pantalla se muestra un listado con los terrenos cargados en la aplicación en forma de tabla con columnas que muestran sus características, tales como su el municipio al que pertenecen, su polígono y su recinto. Además, para cada terreno se incluye una serie de botones en el lado izquierdo de la fila, que permiten la visualización, edición y la eliminación de dicho terreno.

Visor > Terrenos

Terrenos

ID	Municipio	Poligono	Parcela	Recinto	Acciones
bd5b1c42-b5cb-40d6-ad16-0c12b63d1800	Telde	791	382	67	  
a80bfb4d-d312-49a5-a4c1-b63c9147d2fb	Frontera	581	426	305	  
456bd4fa-55a9-41a6-9121-d8ca915d2aed	Frontera	88	987	775	  
19056a29-3b14-4480-8d13-53f1e6251115	Frontera	513	74	267	  
65402b08-74e6-4c51-9e07-cdff85f19610	Frontera	95	127	413	  
f58c722d-0189-4ec8-a126-ace6f3ff2bea	Frontera	338	389	352	  
025eab05-bb98-45d0-874b-6f151fc546e0	Frontera	734	11	568	  
c6d95d50-afbf-4974-9456-c4377954797e	Frontera	5	69	240	  
094dac5f-6d0b-408e-a469-92814ffa8fc6	Frontera	661	710	384	  
e2e4ac27-8f2b-4e95-8baa-50684eea2162	Frontera	234	630	939	  

Items per page: 10 1-10 of 2822 < > >|

CREAR TERRENO

Figura 4.6: Listado de terrenos

Debajo de la tabla se da la opción de añadir nuevos terrenos. Además, en esta vista se presentan algunas métricas que reflejan información sobre el conjunto de los mismos, tales como porcentaje del total de número de terrenos por cada isla o la extensión que abarcan en total, tal y como puede verse en la Figura 4.7.

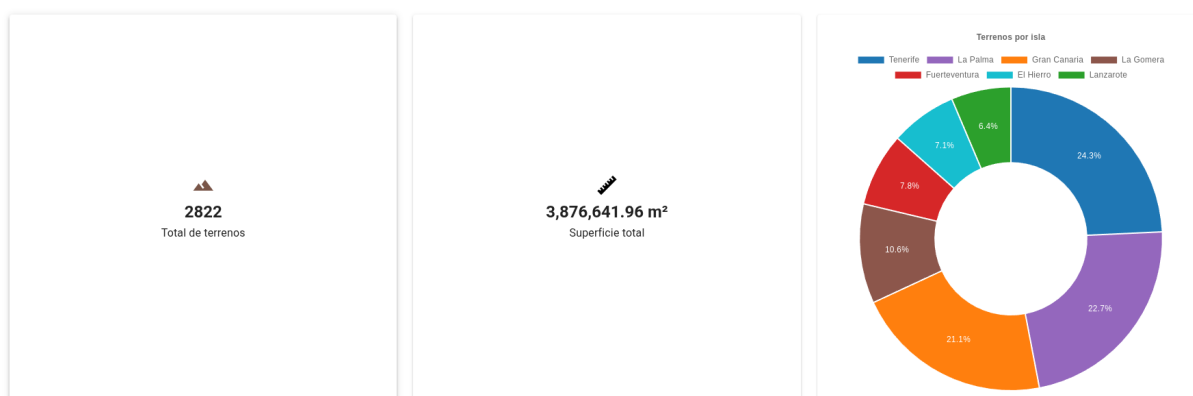


Figura 4.7: Métricas de terrenos

También se presenta un mapa similar al del visualizador de terrenos pero en forma de tarjeta que muestra los terrenos en el que, si se hace click a alguno de ellos, se lleva directamente a la vista de edición del mismo. Esto puede verse en la Figura 4.8.

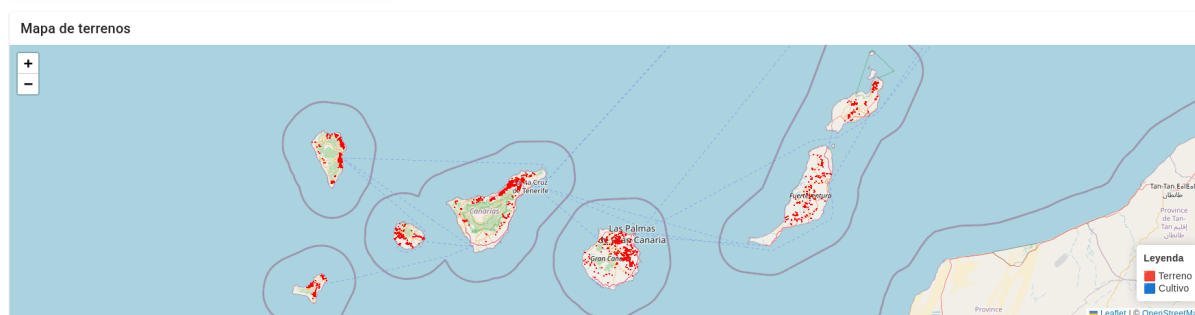


Figura 4.8: Mapa de terrenos

Edición de terrenos

En esta vista se habilita la edición del terreno que se haya seleccionado. Los valores del formulario son modificables y hay una opción para guardar los datos del terreno con sus nuevos valores. Además, en la parte derecha de la vista tal y como puede verse en la Figura 4.9, hay un mapa que muestra la extensión geográfica asociada a dicho terreno y ofrece herramientas para la edición del mismo.



Figura 4.9: Vista de edición de un terreno

Existe también una vista para la creación de terrenos, visualmente igual a esta solo que se parte sin datos y, también para la visualización de la información del cultivo, siendo igual que esta pero sin permitir la edición de los datos.

4.4.4. Gestión de cultivos

Para la gestión de los cultivos se ofrecen herramientas para la disposición de la información y para, de forma similar a los terrenos, poder realizar modificaciones y ajustes si fuera necesario.

Listado de cultivos

Muestra información acerca de todos los cultivos disponibles en la aplicación en forma de tabla (Figura 4.10), permitiendo la modificación, eliminación y consulta de información de los mismos. También permite la creación de nuevos cultivos con el botón inferior de creación.

Visor > Cultivos

Cultivos

ID	Fecha de inicio de cosecha	Fecha de fin de cosecha	Producto	Municipio	Acciones
b2d33afd-7cef-48ad-9679-6c2b09f18d2c	21/12/2024	31/08/2025	Plátano	Villa De Mazo	  
24cf7f63-69c9-4113-b3f4-f5fe7540d8e5	30/05/2025	13/09/2025	Tomate	Valverde	  
a28bb5a2-d603-4184-8adf-ce7bbbf66256	20/04/2025	10/09/2025	Limón	Frontera	  
187b6055-fef4-46ba-a890-2a1cac374194	15/12/2024	12/09/2025	Manzana	El Pinar	  
91e88ac3-7c8c-4a08-87fc-b4e25591d1c5	14/12/2024	14/07/2025	Fresa	Frontera	  
ea723557-1695-4bab-a464-0125b38b704c	24/05/2025	28/07/2025	Uva	El Pinar	  
2fffb2b5-c39a-4565-88e1-47cdc58ae147	17/04/2025	01/08/2025	Cebada	El Pinar	  
1f6311b7-397a-41f0-9ab8-6dfe448a5951	05/04/2025	07/08/2025	Papa	El Pinar	  
52fda2a0-a9b6-4720-af16-37d0961ffb62	03/01/2025	27/09/2025	Uva	El Pinar	  
0de5ccdf-6992-40b0-9d0f-cf105cd92cc5	05/04/2025	22/08/2025	Lechuga	Frontera	  

Items per page: 10 1-10 of 2925 < > >|

CREAR CULTIVO

Figura 4.10: Listado de cultivos

De forma similar al listado de terrenos, se muestran métricas referentes a información de los cultivos cargados, como el porcentaje del total de cultivos repartido entre islas o porcentajes de las cantidades de cada tipo de cultivo (cantidad de tomate, papa, plátano, etc.) que hay cargados en la aplicación entre otros.

También cuenta con un mapa que carga las extensiones geográficas de los cultivos y permite acceder a las opciones de edición al clicar sobre ellos, como puede verse en la Figura 4.11.

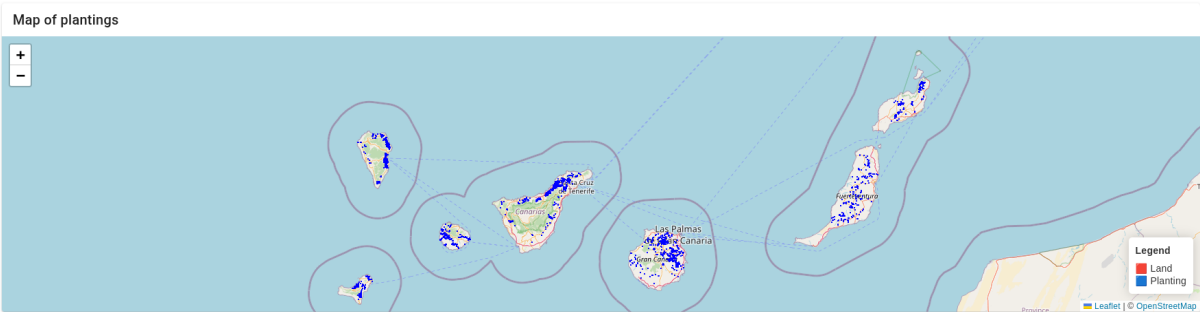


Figura 4.11: Mapa de cultivos

Edición de cultivos

La edición de los cultivos se presenta en una vista que, tal y como se aprecia en la Figura 4.12, dispone los datos del cultivo a modificar en un formulario editable y un mapa con opciones de edición para el polígono que delimita su extensión geográfica. Finalmente, dispone de un botón para poder guardar los cambios.

Visor > Cultivos > Actualizar cultivo

Actualizar cultivo

Cultivo

Fecha de inicio de cosecha *

12/21/2024

Fecha de fin de cosecha *

08/31/2025

Producto *

Plátano

Terreno

f950ddc3-19ed-4169-9ac6-cc9050fe9d44

Extensión (GeoJSON)

```
{
  "type": "Polygon",
  "coordinates": [
    [
      [
        -17.764945458, 28.610317781
      ],
      [
        -17.764898618, 28.610264319
      ],
      [
        -17.764912557, 28.610257971
      ],
      [
        -17.76496116, 28.610235834
      ],
      [
        -17.765044104, 28.610242734
      ],
      [
        -17.765074424, 28.610285076
      ],
      [
        -17.765039229, 28.610304411
      ],
      [
        -17.765006191, 28.610282172
      ],
      [
        -17.764945458, 28.610317781
      ]
      ]
    ]
  ]
}
```

+ GUARDAR

Figura 4.12: Vista de edición de un cultivo

En la aplicación también se ofrecen vistas que permiten únicamente consultar la información del cultivo, visualmente similar pero sin opciones de edición ni de mapa ni de campos del formulario. En la Figura 4.13 puede verse dicha vista. Por otra parte, la vista para añadir un cultivo nuevo es igual a la de edición, solo que inicia vacía.

Visor > Cultivos > Detalles del cultivo

Detalles del cultivo

Cultivo

Número identificador *

b2d33afd-7cef-48ad-9679-6c2b09f18d2c

Fecha de inicio de cosecha *

21/12/2024

Fecha de fin de cosecha *

31/08/2025

Producto *

Plátano

Terreno *

f950ddc3-19ed-4169-9ac6-cc9050fe9d44

Extensión m²

91.45501172411586

Figura 4.13: Vista de información de un cultivo

4.4.5. Gestión de productores

De forma similar a las otras entidades, se ofrecen herramientas que permitan la gestión de los productores, dando opciones para ajustes sobre los productores cargados

del conjunto de datos, realizar nuevas incorporaciones de productores, eliminaciones o mostrar sus datos.

Listado de productores

Esta vista, que se puede ver en la Figura 4.14, cuenta con una tabla que lista a todos los productores de la aplicación, permitiendo el acceso a vistas de edición, visualización y borrado de forma similar a las vistas de listado anteriores. Al final de la tabla hay un botón para la creación de nuevos productores.

Visor > Productores

Productores

ID	Nombre	NIF	Acciones
a2d351de-5640-4c71-8d8b-aae26ec7e12c	Zulema Suárez	89730173A	  
0b485e8e-fdb9-4bb6-b33f-2ebddc0d8adc	Joel Martinez	69748883Y	  
0c5cef46-118a-4ca0-b7a1-5cc3b0ed4e07	VientoFrontera	77152269P	  
00a8bd5f-5c80-478b-a7bb-de80fc32d9d	PraderasIsladelMeridiano	70383182E	  
82257654-9f1c-4abe-b741-ad0b36730a87	Selene Vega	44735766B	  
2aa91bda-0e1c-46fc-b0aa-b89590c3a170	Zulema Rodriguez	84044316Y	  
9d5b6df1-c701-48be-8d23-7bda60c49735	EcoSabinaValverde	27822926L	  
98154e57-79ee-43e3-829f-65bcc4b37e49	TagoroMalpaso	70785793L	  
4f11362f-2373-45c4-b988-89396624a45a	Omar Hernández	25701268B	  
0196416e-fe3f-4786-8932-bb1c5653c431	PraderasOcéano	14439238E	  

Items per page: 10 1-10 of 259 < > >|

CREAR PRODUCTOR

Figura 4.14: Listado de productores

Edición de productores

La edición de los productores se realiza en una vista que tiene un formulario con los atributos a modificar pre-cargados. Esta lista tiene la particularidad de que, como los productores tienen multitud de terrenos asociados, se disponen estos en una tabla editable y con opciones de borrado y adición a los identificadores de los terrenos a los que están asociados, tal y como puede verse en la Figura 4.15.

Actualizar productor

Productor

Nombre *
Selene Vega

NIF *
44735766B

Terrenos

Nuevo terreno +

ID	
456bd4fa-55a9-41a6-9121-d8ca915d2aed	
19056a29-3b14-4480-8d13-53f1e6251115	
27d29e7d-6db8-4a48-a716-84dd56aab160	
03f9d45b-43c7-4d38-a75f-cedadd242e34	
136d4a2a-940c-4683-9d37-4cc79f0b3768	
08a3adae-55ac-40f8-8d58-ad066ab0a5d6	
a1a96ebb-ffc7-4047-8d01-745e6946b934	
ea5c6ef4-2e11-4e74-b74c-6f50e0d65413	

Items per page: 10 1-8 of 8 |< < > >|

GUARDAR

Figura 4.15: Vista de edición de un productor

La vista de creación es igual a la de edición solo que vacía al inicio, mientras que la de consulta de datos varía en el sentido de que, a parte de que los campos no son editables, no se muestran tampoco las opciones de añadir identificadores de cultivos asociados a dicho productor. Además, se muestra también una lista de los cultivos asociados a dichos terrenos.

4.4.6. Gestión de subvenciones

Las gestiones también cuentan con herramientas de visualización en conjunto, edición, adición, eliminación y visión de forma individual. A continuación se detallan dichas herramientas.

Listado de subvenciones

El listado de subvenciones es similar a los anteriores, basado en una tabla que lista las subvenciones de la aplicación tal y como puede verse en la Figura 4.16.

Visor > Subvenciones

Subvenciones

ID	Superficie solicitada	Superficie validada	Acciones
ae1a65ec-9531-4d0f-9cc5-d85fc7def20	4.07	0.65	  
3e9a70f7-47cf-444f-87aa-2cb2eb10ce6c	0.78	3.23	  
d3ff1dd2-f28e-499b-99b7-206d79dbfde1	1.82	4.45	  
0a7c06ee-e470-4f36-8f07-ab576b0f8ed1	4.37	4.64	  
f5bb32b5-16de-4ab1-a096-bb1454032b2f	2.34	1.88	  
bb9bc69d-0d90-4739-8f2d-ae282b65a5bd	2.24	2.83	  
940c26f4-533c-418e-b6d8-7198ab19abee	2.81	2.87	  
9a2b1801-3d54-4164-88bf-9d322995e756	4.48	3.58	  
13ebeca0-ac2a-4cf7-9fa4-7868c7c285fa	1.82	1.58	  
d1b70dce-6503-4a99-bb34-624e996507a4	1.63	3.49	  

Items per page: 10 1-10 of 2925 < > >|

CREAR SUBVENCIÓN

Figura 4.16: Listado de subvenciones

Edición de subvenciones

Tal como puede verse en la Figura 4.17, en la vista de edición se dispone un formulario con los datos de la entidad a editar precargados. La vista de creación es igual pero sin datos precargados y, la de consulta de información sobre la subvención, igual pero sin opción a modificar.

Actualizar subvención

Subvención

Superficie solicitada *
4.07Superficie validada *
0.65Cultivo (UUID) *
b2d33afd-7cef-48ad-9679-6c2b09f18d2c

GUARDAR

Figura 4.17: Vista de edición de una subvención

4.4.7. Panel de control

Esta vista se trata de un panel de control diseñado para poder ofrecer una visión general del estado agrícola en las distintas islas del conjunto de datos cargado en la aplicación a través de una serie de métricas dispuestas en los componentes visuales que pueden verse en la Figura 4.18. Contiene:

- Un componente de tarjetas que recoge métricas globales e importantes sobre el

conjunto de datos cargado. Estas métricas son el número de terrenos, el número de cultivos y el número de productores cargados.

- Un gráfico circular que compara la superficie cultivada y la no cultivada, entendiéndose como no cultivada la diferencia entre la extensión total de terreno y la extensión total de cultivos.
- Un gráfico de barras horizontales que muestra aquellos productores con más cantidad de terrenos, junto con el número de cultivos que tienen plantados en ellos.
- Otro gráfico circular donde se compara la extensión de los 10 cultivos con más extensión del conjunto de datos.
- Un gráfico de líneas donde se muestra qué tanta extensión de cada tipo de cultivo hay cultivada en cada momento del año seleccionado.
- Un panel para poder filtrar los datos de entre las distintas islas, con opción a restablecer esos filtros.

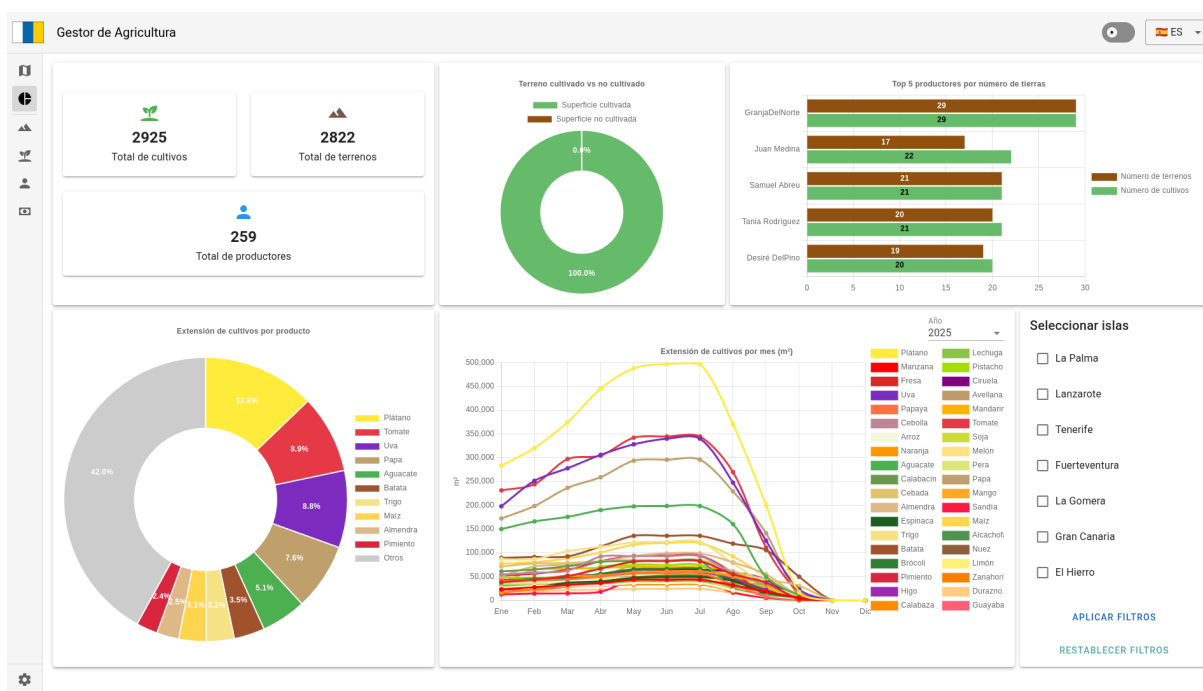


Figura 4.18: Vista del panel de control

Capítulo 5

Experimentación

5.1. Preparación

La aplicación ha sido probada con datos ficticios parcialmente generados a partir de otros reales, ya que en los datos disponibles si bien había información acerca de las extensiones de los cultivos, no se presentaba información acerca de los terrenos.

Es por ello que se han tomado unos datos que contienen información sobre mapa de cultivos de Canarias¹ y, en un primer paso, se han extraído las geometrías que dan forma a los cultivos que contienen. Esto se ha hecho mediante QGIS, importando el Shapefile y, tal y como puede verse en la Figura 5.1 posteriormente, es exportado a GeoJSON, formato con el que se trabaja de ahora en adelante, sacando exclusivamente las geometrías que es la información relevante para la generación de los datos posterior.

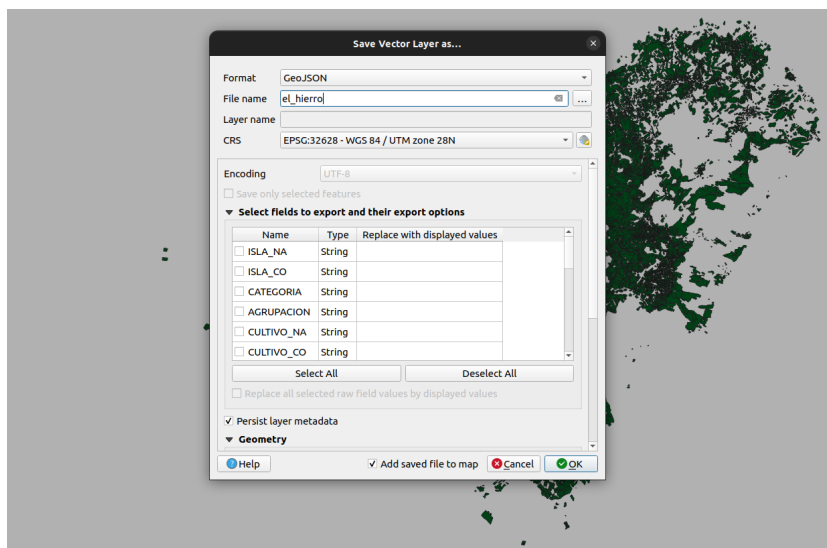


Figura 5.1: Ejemplo de exportación de geometrías de una isla

A partir de estas geometrías, en específico se fija un tope de, por ejemplo, las primeras

¹<https://datos.canarias.es/catalogos/general/dataset/mapa-de-cultivos-de-canarias>

1000 y, por medio de un script de Python, se fusionan todas aquellas que estuvieran juntas y se asume que estas son cultivos que conforman un terreno, obteniéndose dos conjuntos de geometrías finalmente: terrenos y cultivos.

Luego, con el resultado, por medio de un script de Python, se generan terrenos, cultivos, subvenciones y productores con sus relaciones entre ellos, obteniendo así el conjunto de datos con el que trabajar.

Para que la asignación de municipios fuera acorde a dónde se sitúan geográficamente las geometrías, se utiliza un conjunto de datos que contiene las delimitaciones territoriales de los municipios de las islas² y, si la geometría del terreno está contenida en dicha delimitación, se le asigna el nombre del municipio, consiguiendo así unificar también los nombres del conjunto de datos.

También, otro aspecto a tener en cuenta es que, para que los datos tengan un sentido, a la hora de asignar los tipos de cultivos, se han seleccionado una serie de cultivos y se les han asignado unos pesos para variar su aparición entre los cultivos de las distintas islas.

En total para esta prueba, buscando generar un entorno realista y de más estrés para la aplicación, se han generado un total de 5950 cultivos, 5675 terrenos, 526 productores y 5950 subvenciones.

5.2. Resultados

Se comienzan cargando los datasets generados para las distintas islas en el importador de datos de la aplicación tal y como puede verse en la Figura 5.2.

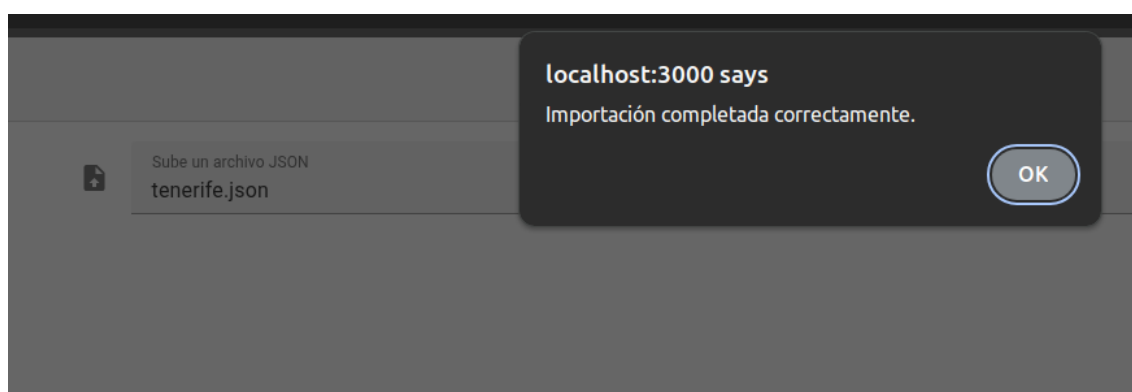


Figura 5.2: Importación de datos a la aplicación

Una vez cargados los datos, es posible visualizarlos en el mapa (Figura 5.3), renderizándose los distintos polígonos con colores en función de su tipo o, también, pueden verse métricas de los datos cargados en el panel de control (Figura 5.4).

²<https://opendata.sitcan.es/dataset/islas-y-municipios/resource/8964f914-7a89-4b01-8780-0696b0ac6>

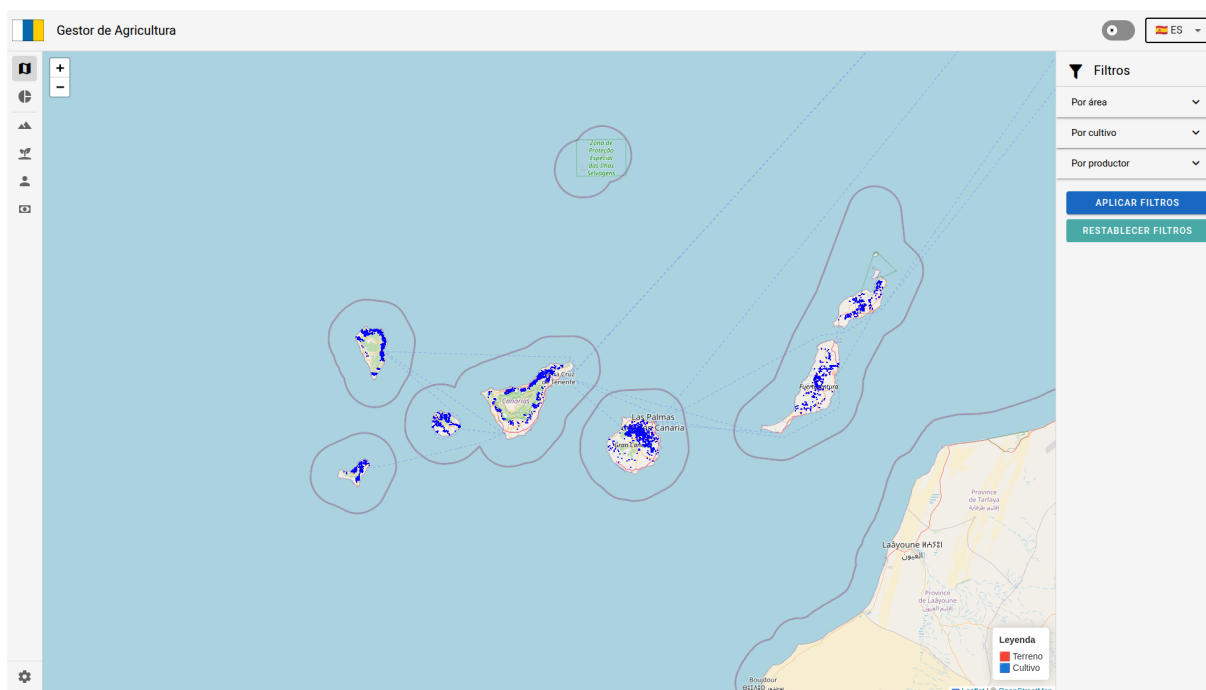


Figura 5.3: Visualización de los datos

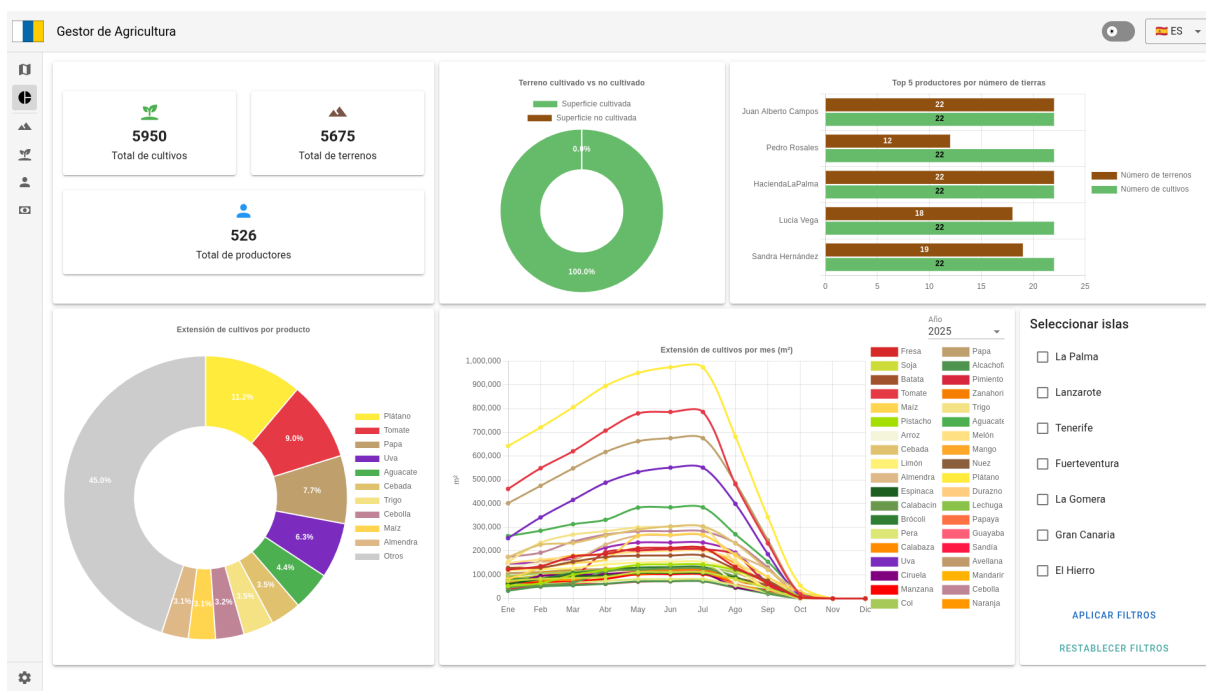


Figura 5.4: Visualización de métricas

Con esto podemos ver el correcto funcionamiento de la aplicación, permitiendo cargar los datos de las distintas islas, mostrarlos sobre el mapa y poder crear métricas de los mismos, dando información sobre indicadores y aspectos clave.

Capítulo 6

Presupuesto

A continuación se muestra información sobre una estimación de costes derivados de la ejecución del proyecto. Son detallados costos asociados a la planificación y la obtención de los requisitos del proyecto, el prototipado, la valoración de las tecnologías a usar, el desarrollo del front-end y back-end, las pruebas, la redacción de la documentación y la experimentación.

Con el objetivo de establecer un valor estimado al precio por hora empleada en el proyecto, se ha sacado información de diversos portales que publican el salario medio de un desarrollador web full-stack junior en España. Para ello, se han consultado Jobted¹, Glassdoor² e Indeed³. Con un cálculo del promedio por hora, se obtiene 14,77 euros la hora, redondeados a un precio de 15 euros la hora.

En la tabla 6.1 se especifican las tareas y las horas invertidas, así como el precio final de cada tarea, el tiempo total y el precio final.

Tarea	Nº Horas	Precio final de la tarea (€)
Planificación y obtención de requisitos	10	150 €
Prototipado	10	150 €
Valoración y elección tecnologías	5	75 €
Desarrollo del front-end	115	1.725 €
Desarrollo del back-end	105	1.525 €
Pruebas	25	375 €
Redacción de documentación	10	150 €
Experimentación	20	300 €
TOTAL	300	4.500 €

Tabla 6.1: Presupuesto

En lo referido a labores de mantenimiento, se estima un presupuesto mensual en la tabla 6.2, que contempla cuestiones derivadas de mantenimiento, corrección de errores y

¹<https://www.jobted.es/salario/full-stack-developer>

²https://www.glassdoor.es/Sueldos/junior-full-stack-developer-sueldo-SRCH_K00,27.htm

³<https://es.indeed.com/career/desarrollador-junior/salaries>

soporte.

Tarea	Nº Horas	Precio de horas al mes (€)
Mantenimiento (mejoras y actualización)	10	150 €
Corrección de errores	10	150 €
Soporte	10	150 €
TOTAL	30	450 € / mes

Tabla 6.2: Presupuesto mensual de mantenimiento

Capítulo 7

Conclusiones y líneas futuras

7.1. Conclusiones

En este Trabajo Final de Grado se ha desarrollado una aplicación full-stack orientada a visualizar la extensión de los distintos cultivos de Canarias con el propósito de ofrecer información visual y relevante sobre estos recursos. Para ello, se implementó el front-end usando Typescript y Vue 3 entre otras tecnologías y, el back-end usando Java y Spring Boot, junto con PostgreSQL como base de datos relacional. La aplicación permite cargar conjuntos de datos para ser visualizados y, además, da la posibilidad de realizar modificaciones a los datos cargados para poder realizar ajustes. También se ofrece un panel de control sobre el que poder ver las métricas de los datos cargados.

Durante el desarrollo del proyecto se encontraron dificultades como por ejemplo la selección y preparación de conjuntos de datos para poder usar la aplicación ya que los conjuntos de datos disponibles no tenían toda la información relevante para mostrar en el trabajo. Esto obligó a usar dichos conjuntos de datos como base para los polígonos de los distintos cultivos y terrenos y, a partir de ellos, generar datos ficticios.

Se enfrentaron desafíos a la hora de modelar la base de datos relacional debido a discrepancias entre cómo se relacionaban las entidades del dominio entre sí y cómo se representarían dichas relaciones en la base de datos. Estas diferencias complicaron significativamente la lógica de acceso y escritura de datos mediante JDBC.

La realización de este Trabajo Final de Grado me ha aportado un aprendizaje incommensurable. He podido aprender de primera mano y profundizar en la Arquitectura Hexagonal. Además, he ganado soltura a la hora de trabajar con herramientas geoespaciales y tecnologías de visualización de datos. En definitiva, el desarrollo de este trabajo me ha permitido ver cómo se conectan todas las piezas de un proyecto real.

7.2. Líneas futuras

Para futuras mejoras del proyecto se propone lo siguiente:

- Mediante el uso de algoritmos de clusterización, permitir la agrupación de los distintos cultivos según distintos parámetros que puedan ayudar a la mejor toma de decisiones.
- Implementar un sistema de grid personalizable en el dashboard donde los componentes del mismo (tablas, gráficas, etc) puedan ser cambiados de posición, eliminados o añadir otros componentes que se deseen desde la interfaz gráfica, similar a un sistema de ventanas.
- Usar técnicas de Inteligencia Artificial y analítica de datos para, en base a los datos disponibles, realizar análisis predictivos sobre los cultivos por ejemplo, sobre futuros usos que vayan a tener los terrenos, o bien, predecir cambios de tendencias de cultivos en zonas.
- Realizar pruebas con datos reales para poder obtener una mejor retroalimentación sobre el desempeño de la aplicación.

Capítulo 8

Conclusions and Future Work

8.1. Conclusions

In this Final Degree Project, a full-stack application was developed to visualize the extent of the different plantings in the Canary Islands with the aim of providing visual and relevant information about these resources. To achieve this, the front-end was implemented using Typescript and Vue 3, among other technologies, and the back-end using Java and Spring Boot, along with PostgreSQL as a relational database. The application allows uploading datasets for visualization and also offers the ability to make modifications to the uploaded data to make adjustments. A dashboard is also provided to view the metrics of the uploaded data.

During the development of the project, some difficulties were encountered, such as the selection and preparation of datasets for use with the application, since the available datasets did not have all the relevant information to display in the project. This required using these datasets as the basis for the polygons of the different planting and land areas and, from them, generating fictitious data.

Challenges were faced when modeling the relational database due to discrepancies between how domain entities were related to each other and how those relationships would be represented in the database. These differences significantly complicated the logic of accessing and writing data using JDBC.

Completing this Final Degree Project has provided me with immeasurable learning. I was able to learn firsthand and go deeper into Hexagonal Architecture. In addition, I have gained confidence in working with geospatial tools and data visualization technologies. Ultimately, developing this project has allowed me to see how all the pieces of a real project connect.

8.2. Future Lines

For future improvements to the project, the following are proposed:

- By using clustering algorithms, allow the grouping of different plantings according to different parameters that can help in better decision-making.
- Implement a customizable grid system in the dashboard where components (tables, graphs, etc.) can be repositioned, deleted, or other desired components can be added from the graphical interface, similar to a window system
- Use artificial intelligence and data analytics techniques to, based on available data, perform predictive analysis of plantings, for example, on future land uses, or predict changes in planting trends in certain areas.
- Do tests with real data to obtain better feedback on the application's performance.

Bibliografía

- [1] Estadística agraria y pesquera de canarias 2021. Technical report, Consejería de Agricultura, Ganadería y Pesca, 2022.
- [2] H. Butler, M. Daly, A. Doyle, Sean Gillies, T. Schaub, and Stefan Hagen. The GeoJSON Format. RFC 7946, August 2016.
- [3] Julio Costa. Openstreetmap: el mapa libre del mundo. *BITS*, page 46, 1961.
- [4] Paul Crickard and Pratyush Mohanta. *Leaflet.js essentials : create interactive, mobile-friendly mapping applications using the incredibly light yet powerful Leaflet.js platform*. Community Experience Distilled. Packt Publishing, Birmingham, England, 1st edition edition, 2014 - 2014.
- [5] Andrew Cutts. *QGIS quick start guide : a beginner's guide to getting started with QGIS 3.4*. Packt, Birmingham ;, 1st edition edition, 2019.
- [6] Dixon Fabián Flórez Delgado and Deisy Katherine Fernández García. Los sistemas de información geográfica. una revisión. *Revista Facultad de Ciencias Agropecuarias-FAGROPEC*, 9(1):11–16, 2017.
- [7] Michael Dorman. Geojson. In *Introduction to Web Mapping*, pages 169–197. CRC Press, 1 edition, 2020.
- [8] Yurena González González and Adrián García Perdigón. Datos y reflexiones sobre el sector agrario canario. *Revista Atlántida (La Laguna)*, 1(15), 2024.
- [9] Mordechai Haklay and Patrick Weber. Openstreetmap: User-generated street maps. *IEEE Pervasive Computing*, 7(4):12–18, 2008.
- [10] Emilio Ortega Pérez, Belén Martín Ramos, Alejandra Ezquerro Canalejo, and I. Otero. *Sistemas de información geográfica : teoría y práctica*. Dextra Editorial, Madrid, 2016.
- [11] Gabriel Svennerberg. *Beginning Google Maps API 3*. Expert's voice in Web development Beginning Google Maps API 3. Apress, Berkeley, CA, 2nd ed. 2010. edition, 2010.
- [12] Davi Vieira. *Designing hexagonal architecture with Java : an architect's guide to building maintainable and change-tolerant applications with Java and Quarkus*. Packt Publishing, Birmingham ;, 2022.